

VIGIL-AXIS: The Immutable Forensic OS

Automated. Verifiable. BSA 2026 Compliant.

Project Team:

👤 Person Tanmay Verma

👤 Person Deepali Singh

👤 Person Kashish

SYNOPSIS

Introduction

The exponential growth of digital technologies has significantly increased the scale and complexity of cybercrimes. From data breaches and ransomware attacks to insider threats and cyber espionage, modern digital crimes leave behind large volumes of volatile and persistent data. The process of collecting, preserving, and presenting this data as admissible evidence in a court of law constitutes the core of digital forensics. However, the credibility of digital evidence is heavily dependent on the chain of custody, which ensures that the evidence remains untampered from the moment of acquisition to its presentation in court. Traditional digital forensic methodologies rely on manual logging, centralized storage systems, and fragmented tools, which introduce multiple points of failure. These include human errors, unauthorized modifications, and lack of transparency, ultimately leading to questions regarding the authenticity of evidence. With the introduction of the Bharatiya Sakshya Adhiniyam (BSA) 2026, the legal standards for digital evidence in India have become more stringent. The law mandates the inclusion of a Section 63(4) Certificate of Integrity, which requires verifiable proof that digital evidence has not been altered. Existing forensic tools are not inherently designed to meet these compliance requirements, creating a critical gap between technical investigation processes and legal admissibility. To address these challenges, this project proposes VIGIL-AXIS, an advanced digital forensic system that integrates automated evidence collection, cryptographic hashing, and blockchain-based notarization. The system is designed to transform digital forensics from a trust-based process into a mathematically verifiable framework, ensuring integrity, immutability, and legal compliance.

2. Problem Statement

Despite advancements in digital forensic tools, several fundamental issues persist in current investigation workflows:

2.1 Fragile Chain of Custody

In traditional systems, evidence handling relies on manual documentation and centralized databases. These methods are susceptible to:

- Unauthorized access and tampering
- Lack of real-time verification mechanisms
- Inability to provide cryptographic proof of integrity

As a result, even valid evidence can be challenged in court due to insufficient proof of authenticity.

2.2 Volatile Data Loss

Critical forensic artifacts such as:

- RAM contents
- Active network connections
- System logs

are highly volatile and can be lost during system shutdown or delayed acquisition. Most forensic tools operate on dead forensics, where the system is analyzed after being powered off, leading to irreversible loss of valuable evidence.

2.3 Legal Inadmissibility

Under BSA 2026, digital evidence must be accompanied by a verifiable certificate ensuring its integrity. Existing tools:

- Do not automatically generate legal compliance certificates
- Lack transparent audit trails
- Fail to bridge the gap between technical output and judicial requirements

2.4 Fragmented Tool Ecosystem

Investigators often use multiple tools for:

- Evidence collection
- Analysis
- Reporting

This fragmentation creates "gaps" in the chain of custody, making it difficult to ensure continuity and traceability.

2.5 Lack of Transparency and Trust

Centralized systems rely on administrative control, making logs vulnerable to:

- Insider manipulation
- Data deletion
- Unauthorized modification

This undermines the credibility of forensic evidence.

3. Objectives of the Project

The primary objective of VIGIL-AXIS is to establish a secure, automated, and legally compliant digital forensic framework. The specific objectives are:

1. To automate the process of evidence collection from live systems, ensuring minimal data loss and rapid acquisition of volatile artifacts.
2. To ensure data integrity using cryptographic techniques, specifically SHA-256 hashing, which generates unique digital fingerprints for each piece of evidence.
3. To implement blockchain-based notarization, providing an immutable and tamper-proof record of evidence.
4. To establish a robust chain of custody mechanism, tracking every interaction with the evidence.
5. To achieve compliance with BSA 2026, including automated generation of Section 63(4) certificates.
6. To develop a unified platform that integrates evidence collection, storage, verification, and reporting.
7. To enhance transparency and trust through decentralized and verifiable systems.

4. Scope of the Project

The scope of VIGIL-AXIS extends across multiple domains of digital forensics and cybersecurity:

- Cybercrime Investigation: Assisting law enforcement agencies in handling digital evidence.
- Incident Response: Enabling rapid triage and analysis during security breaches.
- Corporate Forensics: Supporting organizations in internal investigations.
- Judicial Systems: Providing verifiable evidence for court proceedings.

The system focuses on:

- Live forensic data acquisition

- Secure storage and retrieval
- Blockchain-based verification
- Legal compliance automation

However, the project does not aim to replace existing forensic analysis tools but rather augment them by providing a secure and verifiable backbone.

5. Proposed System Overview

VIGIL-AXIS introduces a multi-layered architecture designed to ensure security, scalability, and reliability.

5.1 Evidence Collection Layer

- Automated scripts (Python/Bash) extract volatile and persistent artifacts.
- Data includes RAM dumps, logs, and system metadata.
- Ensures rapid acquisition before data loss occurs.

5.2 Hashing and Integrity Layer

- Each artifact is processed using SHA-256 hashing.
- Generates a unique digital fingerprint.
- Ensures that any modification in data can be detected.

5.3 Blockchain Notarization Layer

- Hashes are anchored to a blockchain using smart contracts.
- Provides: Immutable storage, Timestamping, Public verification

5.4 Storage Layer

- Uses decentralized storage (IPFS) for secure data handling.
- Metadata stored in relational databases (PostgreSQL).

5.5 Application Layer

- Investigator dashboard for case management.
- Verification portal for checking evidence authenticity.
- Reporting module for generating legal documents.

6. Methodology

The system follows a structured forensic lifecycle:

8. **Extraction** Automated scripts collect evidence from the target system.
9. **Hashing** Data is converted into SHA-256 hashes.
10. **Notarization** Hashes are recorded on the blockchain via smart contracts.
11. **Storage** Evidence is securely stored in decentralized storage systems.
12. **Verification** Integrity is validated by comparing hashes.
13. **Certification** Legal certificates are generated for court submission.

7. Hardware and Software Requirements

7.1 Hardware Requirements

- Processor: Intel i5 / Ryzen 5 or higher
- RAM: Minimum 8 GB
- Storage: 256 GB SSD or higher
- Network connectivity for blockchain interaction

7.2 Software Requirements

- Operating System: Windows/Linux
- Backend: Flask / Node.js
- Frontend: React + Vite
- Database: PostgreSQL
- Blockchain: Solidity, Hardhat
- Storage: IPFS
- Security: JWT authentication

8. Expected Outcomes

The implementation of VIGIL-AXIS is expected to achieve:

- A tamper-proof chain of custody
- Reduced human error in forensic processes

- Real-time evidence acquisition
- Enhanced legal admissibility
- Increased trust in digital forensic systems

9. Conclusion of Synopsis

The proposed system, VIGIL-AXIS, addresses critical limitations in existing digital forensic methodologies by integrating automation, cryptographic security, and blockchain technology. By ensuring data integrity, transparency, and compliance with modern legal frameworks such as BSA 2026, the system has the potential to redefine the standards of digital evidence handling. It transforms forensic investigation from a trust-dependent process into a verifiable and scientifically robust framework, thereby strengthening the credibility of digital evidence in judicial systems.

ABSTRACT

The exponential growth of digital technologies has led to a corresponding increase in cybercrimes, making digital forensics a critical domain in modern investigative processes. However, traditional digital forensic systems face significant challenges related to data integrity, chain of custody, and legal admissibility. Evidence collected from digital sources is often susceptible to tampering, unauthorized access, and lack of verifiable authenticity, which can compromise its reliability in judicial proceedings. Addressing these challenges requires a system that ensures secure evidence handling, transparency, and compliance with legal standards.

This project, titled “**VIGIL-AXIS: The Immutable Forensic Operating System,**” proposes a comprehensive solution that integrates blockchain technology with digital forensic workflows to enhance the security, integrity, and traceability of digital evidence. The system is designed as a full-stack application that automates the processes of evidence collection, hashing, storage, verification, and reporting within a unified platform.

The proposed system employs **SHA-256 cryptographic hashing** to generate a unique digital fingerprint for each piece of evidence. This ensures that even the smallest modification in the data can be detected. The generated hash values are then securely stored on a blockchain network using smart contracts, providing immutability and decentralized verification. To address storage challenges, the system utilizes the **InterPlanetary File System (IPFS)** for decentralized and secure evidence storage, while metadata is maintained in a structured database for efficient retrieval.

VIGIL-AXIS also incorporates a role-based access control mechanism, distinguishing between forensic investigators and auditors. Investigators are responsible for evidence acquisition and management, while auditors perform independent verification to ensure data integrity. The system includes a **Verification Engine** that compares locally computed hashes with blockchain-stored hashes, enabling instant detection of tampering. Additionally, an automated **Audit Trail** maintains a detailed log of all system activities, ensuring a continuous and verifiable chain of custody.

One of the key features of the system is the generation of legally compliant reports, including a **BSA 2026 admissibility certificate**, which certifies that the digital evidence meets integrity and authenticity requirements for court submission. This significantly reduces the gap between technical forensic processes and legal standards.

The implementation of VIGIL-AXIS demonstrates that integrating blockchain with digital forensics can significantly enhance trust, transparency, and reliability in evidence handling. The system not only improves the efficiency of forensic investigations but also strengthens the credibility of digital evidence in legal contexts. As cyber threats continue to evolve, solutions like VIGIL-AXIS provide a scalable and future-ready approach to secure digital forensic operations.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this project titled **“VIGIL-AXIS: The Immutable Forensic Operating System.”**

First and foremost, I extend my heartfelt thanks to my project guide, whose invaluable guidance, continuous support, and constructive feedback have played a crucial role throughout the development of this project. Their insights and encouragement helped me navigate both technical and conceptual challenges effectively.

I would also like to express my gratitude to the faculty members of the department for providing the necessary academic foundation and resources required for this work. Their teachings have significantly contributed to my understanding of the subject and its practical applications.

I am thankful to my institution, **Dronacharya College of Engineering**, for offering the infrastructure, tools, and environment necessary to carry out this project successfully.

I would also like to acknowledge my friends and peers for their constant support, suggestions, and motivation during the course of this project. Their discussions and feedback were extremely helpful in refining various aspects of the system.

Finally, I would like to express my deepest appreciation to my family for their unwavering support, encouragement, and belief in my abilities, which kept me motivated throughout this journey.

TABLE OF CONTENTS

Section	Topic
FRONT MATTER	
	SYNOPSIS
	ABSTRACT
	ACKNOWLEDGEMENT
CHAPTER 1	INTRODUCTION
1.1	Overview of Digital Forensics
1.2	About the Project
1.3	Existing Problem Scenario
	1.3.1 Fragile Chain of Custody
	1.3.2 Loss of Volatile Data
	1.3.3 Legal Inadmissibility of Evidence
	1.3.4 Fragmented Forensic Tools
	1.3.5 Lack of Transparency and Trust
1.4	Objectives of the Project
	1.4.1 Automation of Evidence Collection
	1.4.2 Ensuring Data Integrity
	1.4.3 Blockchain-Based Immutability

	1.4.4 Establishing a Robust Chain of Custody
	1.4.5 Legal Compliance (BSA 2026)
	1.4.6 Integration of Forensic Processes
	1.4.7 Enhancing Trust and Transparency
1.5	Proposed System Model
	1.5.1 Evidence Collection Layer
	1.5.2 Processing and Hashing Layer
	1.5.3 Blockchain Layer
	1.5.4 Storage Layer
	1.5.5 Application Layer
1.6	Software Requirements
1.7	Hardware Requirements
1.8	Summary
CHAPTER 2	SYSTEM DESIGN
2.1	Introduction
2.2	System Architecture Design
	2.2.1 Evidence Collection Layer
	2.2.2 Processing and Hashing Layer
	2.2.3 Blockchain (Trust & Immutability Layer)
	2.2.4 Storage Layer

	2.2.5 Application Layer
2.3	Data Flow Diagram (DFD)
	2.3.1 Explanation of DFD Components
2.4	Entity Relationship Diagram (ERD)
	2.4.1 Entities Description
	2.4.2 Relationships Summary
2.5	Use Case Diagram
	2.5.1 Actors
	2.5.2 Use Cases
	2.5.3 Description
2.6	Activity Diagram
	2.6.1 Workflow Explanation
2.7	Design Considerations
	2.7.1 Security
	2.7.2 Scalability
	2.7.3 Reliability
	2.7.4 Legal Compliance
2.8	Summary
CHAPTER 3	IMPLEMENTATION
3.1	Introduction

3.2	Development Environment
	3.2.1 Software Tools Used
	3.2.2 System Configuration
3.3	System Modules
3.4	Forensic Investigator
	3.4.1 Secure Access Gateway (Login – Investigator Role)
	3.4.2 Case Command Dashboard (Investigator)
	3.4.3 Case Management (Folder View)
	3.4.4 Case Management (Detailed View)
	3.4.5 Evidence Lab (Upload Module)
	3.4.6 Audit Trail
3.5	Forensic Auditor
	3.5.1 Secure Access Gateway (Auditor Role)
	3.5.2 Case Command Dashboard (Auditor)
	3.5.3 Verification Engine (Pending State)
	3.5.4 Verification Engine (Successful Case)
	3.5.5 BSA 2026 Admissibility Certificate
	3.5.6 Verification Engine (Tampered Case)
	3.5.7 Immutable Ledger
3.6	Code Snapshots

	3.6.1 Backend Code (Verification Logic)
	3.6.2 Frontend Logic (React Hooks)
	3.6.3 Smart Contract (Blockchain Layer)
3.7	Security Implementation
3.8	System Workflow Implementation
3.9	Integration of Modules
3.10	Security Implementation
3.11	Challenges Faced
3.12	Summary
CHAPTER 4	TESTING
4.1	Introduction
4.2	Testing Objectives
4.3	Types of Testing Performed
4.4	Unit Testing
	4.4.1 Overview
	4.4.2 Unit Testing of Modules
4.5	Integration Testing
	4.5.1 Overview
	4.5.2 Integration Scenarios
	4.5.3 Results

4.6	System Testing
	4.6.1 Overview
	4.6.2 Functional Testing
	4.6.3 Non-Functional Testing
4.7	Security Testing
	4.7.1 Overview
	4.7.2 Security Measures Tested
	4.7.3 Test Results
4.8	Performance Testing
	4.8.1 Overview
	4.8.2 Metrics
	4.8.3 Results
4.9	User Acceptance Testing (UAT)
	4.9.1 Overview
	4.9.2 Feedback
4.10	Test Data and Results
4.11	Error Handling and Debugging
4.12	Limitations of Testing
4.13	Summary
CHAPTER 5	TECHNICAL STACK & INNOVATION

CHAPTER 6	FINANCIAL ANALYSIS
6.1	Introduction
6.2	Cost Structure
	6.2.1 Initial Setup
	6.2.2 Operational Cost per Case
6.3	Revenue Model
6.4	Market Analysis
	6.4.1 Total Addressable Market (TAM)
	6.4.2 Serviceable Available Market (SAM)
	6.4.3 Serviceable Obtainable Market (SOM)
6.5	Profitability Analysis
6.6	Growth Projection
6.7	Cost-Benefit Analysis
6.8	Break-Even Analysis
6.9	Financial Feasibility
6.10	Summary
CHAPTER 7	CONCLUSION
7.1	Introduction
7.2	Summary of the Work
7.3	Achievement of Objectives

7.4	Key Contributions
7.5	Advantages of the System
7.6	Limitations
7.7	Conclusion
CHAPTER 8	FUTURE SCOPE
8.1	Introduction
8.2	Technical Enhancements
	8.2.1 AI-Based Evidence Analysis
	8.2.2 Advanced Blockchain Integration
	8.2.3 Multi-Chain Support
	8.2.4 Enhanced Security Features
8.3	Functional Enhancements
	8.3.1 Mobile Application Development
	8.3.2 Cloud Integration
	8.3.3 Real-Time Collaboration
8.4	Legal and Compliance Expansion
8.5	Industry Applications
8.6	Research Opportunities
8.7	Conclusion

REFERENCES	REFERENCES
9.1	Introduction
9.2	Books
9.3	Research Papers
9.4	Websites
9.5	Tools and Documentation
9.6	Standards and Legal References
9.7	Conclusion

LIST OF FIGURES

Figure No.	Description / Title
CHAPTER 1	
Figure 1.1	Proposed System Model (Layered Architecture)
CHAPTER 2	
Figure 2.1	System Architecture Design
Figure 2.2	Data Flow Diagram (DFD) - Level 1
Figure 2.3	Entity Relationship Diagram (ERD)
Figure 2.4	Use Case Diagram
Figure 2.5	Activity Diagram (Forensic Workflow)
CHAPTER 3	
Figure 3.1	Secure Access Gateway (Investigator Login)
Figure 3.2	Case Command Dashboard (Investigator)
Figure 3.3	Case Management (Folder View)
Figure 3.4	Case Management (Detailed View)
Figure 3.5	Evidence Lab (Upload Module)
Figure 3.6	Audit Trail View
Figure 3.7	Secure Access Gateway (Auditor Login)

Figure 3.8	Case Command Dashboard (Auditor)
Figure 3.9	Verification Engine (Pending State)
Figure 3.10	Verification Engine (Successful Case - Verified)
Figure 3.11	BSA 2026 Admissibility Certificate
Figure 3.12	Verification Engine (Tampered Case - Integrity Alert)
Figure 3.13	Immutable Ledger View (Blockchain Records)
Figure 3.14	Backend Code Snippet (Verification Logic)
Figure 3.15	Frontend Code Snippet (React Hooks)
Figure 3.16	Smart Contract Code (Solidity)
CHAPTER 5	
Figure 5.1	Technical Stack and Innovation Overview

LIST OF TABLES

Table No.	Description / Title
CHAPTER 4	
Table 4.1	Unit Testing: Evidence Collection Module
Table 4.2	Unit Testing: Hash Generation Module
Table 4.3	Unit Testing: Blockchain Module
Table 4.4	Unit Testing: Verification Module
Table 4.5	Integration Scenario 1: Collection -> Hashing -> Blockchain
Table 4.6	Integration Scenario 2: Storage -> Verification -> Reporting
Table 4.7	Functional Testing Results
Table 4.8	Security Testing Scenarios and Results
Table 4.9	Sample Test Data and Observed Verification Results
CHAPTER 6	
Table 6.1	Initial Setup Cost Structure
Table 6.2	Operational Cost per Case
Table 6.3	Profitability Analysis (Example Calculation)

CHAPTER 1: INTRODUCTION

VIGIL-AXIS: The Immutable Forensic OS

1.1 Overview of Digital Forensics

The rapid digitization of modern society has led to an exponential increase in the generation, storage, and transmission of digital data. While this transformation has improved efficiency across industries, it has simultaneously created new avenues for cybercrime. Digital forensics has emerged as a critical discipline that focuses on the identification, acquisition, preservation, analysis, and presentation of digital evidence in a legally admissible manner.

Digital forensic investigations typically involve examining computing devices such as desktops, laptops, mobile phones, and network infrastructures. The primary objective is to reconstruct events, identify malicious activities, and provide credible evidence that can be presented in courts of law. However, the reliability of digital evidence is fundamentally dependent on maintaining its integrity, authenticity, and traceability throughout the investigation lifecycle.

One of the most critical components of digital forensics is the chain of custody, which ensures that evidence is handled in a controlled and documented manner from the moment of acquisition to its presentation in court. Any break or inconsistency in this chain can render the evidence inadmissible.

1.2 About the Project

The proposed system, VIGIL-AXIS: The Immutable Forensic OS, is designed to address the inherent limitations of traditional digital forensic methodologies by integrating automation, cryptographic techniques, and blockchain technology. VIGIL-AXIS introduces a unified platform that automates the process of evidence collection, generates cryptographic fingerprints using SHA-256 hashing, and anchors these fingerprints onto a decentralized blockchain ledger. This approach ensures that once evidence is collected, its integrity can be verified at any point in time without relying on trust in a centralized authority.

The system is specifically designed to comply with the Bharatiya Sakshya Adhiniyam (BSA) 2026, which mandates strict standards for digital evidence admissibility, including the

requirement of a Section 63(4) Certificate of Integrity. VIGIL-AXIS bridges the gap between technical forensic processes and legal requirements by automating the generation of such certificates.

Key functionalities of the system include:

- Automated extraction of volatile and persistent forensic artifacts
- Generation of SHA-256 hashes for data integrity
- Blockchain-based notarization for immutability
- Decentralized storage using IPFS
- Real-time verification of evidence authenticity
- Audit trail for complete chain-of-custody tracking

The system transforms digital forensics from a process based on human trust into a mathematically verifiable framework, thereby enhancing credibility and reliability.

1.3 Existing Problem Scenario

Despite the availability of various digital forensic tools, several critical challenges remain unresolved. These challenges significantly impact the effectiveness and reliability of forensic investigations.

1.3.1 Fragile Chain of Custody

Traditional forensic processes rely heavily on manual documentation and centralized databases to maintain the chain of custody. This approach is inherently vulnerable to:

- Human errors during documentation
- Unauthorized access or modification
- Lack of real-time verification

Since centralized systems can be altered by administrators or attackers, it becomes difficult to prove that the evidence has remained unchanged. This creates serious doubts about the authenticity of evidence in legal proceedings.

1.3.2 Loss of Volatile Data

Volatile data refers to information stored in temporary memory, such as RAM, active processes, and network connections. This data is lost when the system is powered off. Most forensic tools

follow a dead forensics approach, where analysis begins after shutting down the system. As a result:

- Critical runtime evidence is lost
- Attack traces disappear
- Investigation accuracy is reduced

There is a lack of efficient systems capable of performing live forensic acquisition in an automated manner.

1.3.3 Legal Inadmissibility of Evidence

Under the BSA 2026, digital evidence must be accompanied by verifiable proof of integrity. This includes:

- Proof that data has not been altered
- Traceability of all interactions
- Certification of authenticity

Existing tools do not automatically generate such legal certificates, making it difficult for investigators to meet compliance requirements. Consequently, valid evidence may be rejected in court due to procedural shortcomings.

1.3.4 Fragmented Forensic Tools

Investigators often rely on multiple tools for different tasks:

- Data acquisition
- Analysis
- Reporting

This fragmented ecosystem leads to inconsistent workflows, gaps in chain-of-custody tracking, and increased complexity. There is no unified system that integrates all stages of the forensic lifecycle.

1.3.5 Lack of Transparency and Trust

Centralized logging systems are prone to manipulation. Since administrators have control over the system, logs can be modified or deleted, and unauthorized access may go undetected. This undermines the credibility of forensic investigations.

1.4 Objectives of the Project

The primary goal of VIGIL-AXIS is to develop a secure, automated, and legally compliant digital forensic system. The specific objectives are as follows:

1.4.1 Automation of Evidence Collection

To design a system capable of automatically collecting forensic artifacts from live systems, ensuring minimal loss of critical data.

1.4.2 Ensuring Data Integrity

To implement cryptographic hashing (SHA-256) for generating unique digital fingerprints of evidence.

1.4.3 Blockchain-Based Immutability

To utilize blockchain technology for storing evidence hashes in a tamper-proof and immutable manner.

1.4.4 Establishing a Robust Chain of Custody

To maintain a transparent and traceable record of all interactions with evidence.

1.4.5 Legal Compliance (BSA 2026)

To automate the generation of legally valid certificates for digital evidence.

1.4.6 Integration of Forensic Processes

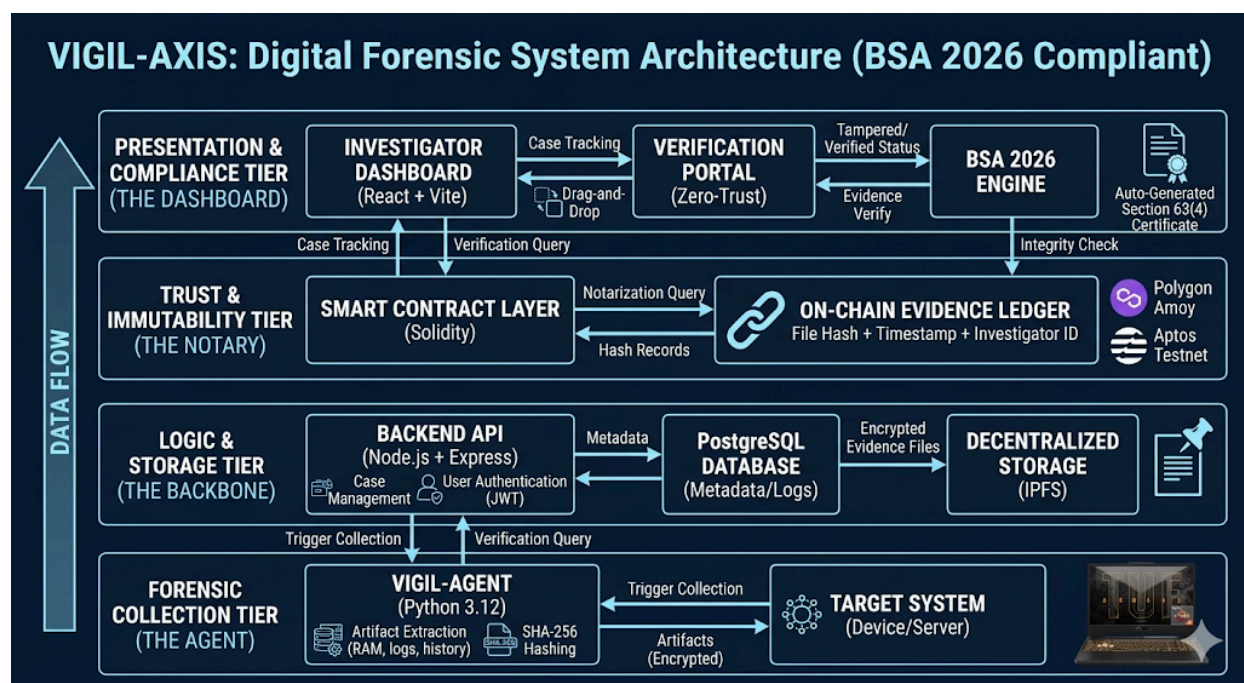
To create a unified platform that integrates collection, storage, verification, and reporting.

1.4.7 Enhancing Trust and Transparency

To eliminate reliance on human trust by providing mathematically verifiable proof of integrity.

1.5 Proposed System Model

The VIGIL-AXIS system follows a layered architectural model, ensuring modularity, scalability, and security.



1.5.1 Evidence Collection Layer

- Automated scripts (Python/Bash) extract data from target systems
- Captures volatile data such as RAM and logs
- Ensures rapid and efficient acquisition

1.5.2 Processing and Hashing Layer

- Data is processed and converted into SHA-256 hashes
- Each hash uniquely represents the original data
- Even minor changes result in different hash values

1.5.3 Blockchain Layer

Smart contracts store hash values on the blockchain. Provides:

- Immutability
- Timestamping
- Public verification

1.5.4 Storage Layer

- Evidence stored using IPFS (decentralized storage)
- Metadata stored in relational databases (PostgreSQL)

- Ensures redundancy and security

1.5.5 Application Layer

- Investigator dashboard
- Verification portal
- Reporting system

1.6 Software Requirements

The software stack is designed to ensure performance, scalability, and security.

Backend

- Flask / Node.js
- RESTful API architecture
- Handles data processing and communication

Frontend

- React with Vite
- Interactive user interface
- Dashboard visualization

Database

- PostgreSQL
- Stores metadata, logs, and user information

Blockchain

- Solidity for smart contracts
- Hardhat for development and testing
- Polygon/Aptos for deployment

Storage

- IPFS (InterPlanetary File System)
- Decentralized and secure data storage

Security Tools

- JWT Authentication
- SHA-256 hashing

1.7 Hardware Requirements

To ensure optimal system performance, the following hardware specifications are required:

Minimum Requirements

- Processor: Intel i5 / Ryzen 5
- RAM: 8 GB
- Storage: 256 GB SSD

Recommended Requirements

- Processor: Intel i7 / Ryzen 7
- RAM: 16 GB
- Storage: 512 GB SSD

Additional Requirements

- Stable internet connection
- Blockchain node access

1.8 Summary

This chapter introduced the concept of digital forensics and highlighted the limitations of existing systems, including fragile chain-of-custody mechanisms, loss of volatile data, and lack of legal compliance. The proposed system, VIGIL-AXIS, addresses these challenges by integrating automation, cryptographic hashing, and blockchain technology into a unified platform. By ensuring data integrity, transparency, and compliance with modern legal frameworks such as BSA 2026, the system aims to revolutionize the way digital evidence is handled. The subsequent chapters will provide a detailed discussion of system design, implementation, testing, and results.

CHAPTER 2: SYSTEM DESIGN

2.1 Introduction

System design is a crucial phase in the software development lifecycle that translates conceptual requirements into a structured and implementable architecture. In digital forensic systems, design plays an even more critical role due to the need for data integrity, traceability, security, and legal compliance. The design of VIGIL-AXIS follows a layered and modular architecture, ensuring:

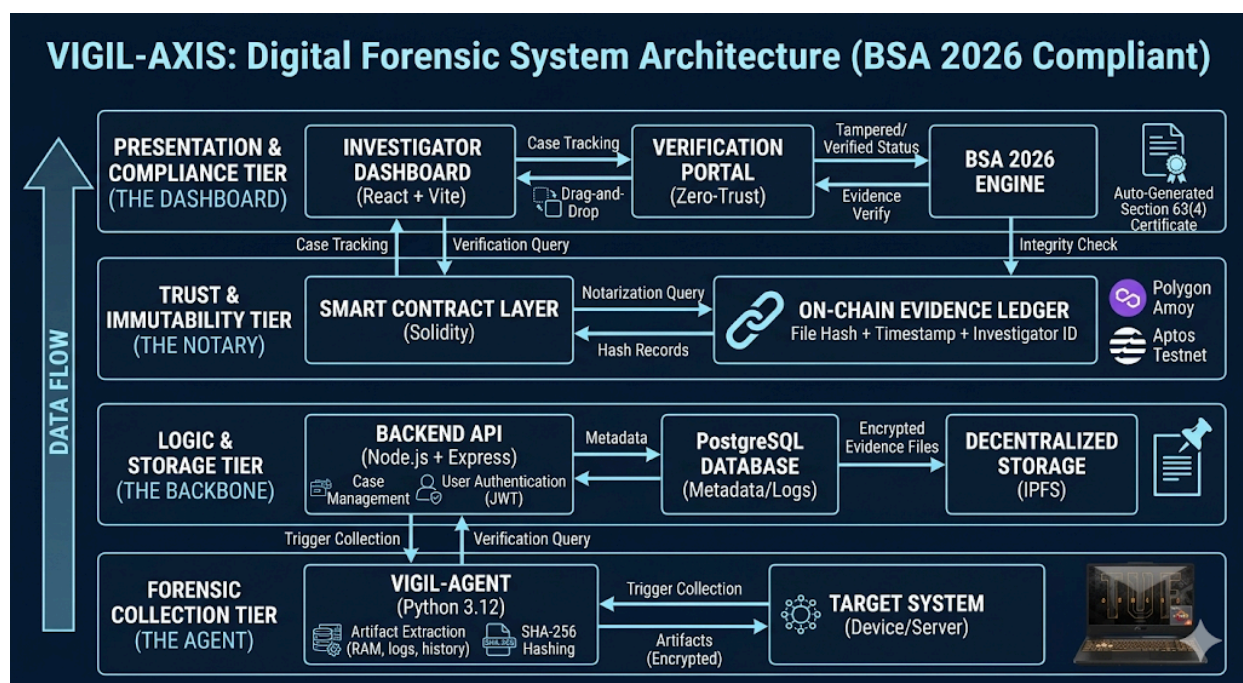
- Separation of functional responsibilities
- Scalability and flexibility
- High security and tamper resistance
- Compliance with legal standards (BSA 2026)

The system integrates forensic data acquisition, cryptographic hashing, blockchain notarization, and verification mechanisms into a unified workflow.

2.2 System Architecture Design

The architecture of VIGIL-AXIS is divided into multiple layers:

1. Evidence Collection Layer
2. Processing & Hashing Layer
3. Blockchain (Trust) Layer
4. Storage Layer
5. Application (Presentation) Layer



2.2.1 Evidence Collection Layer

This layer is responsible for acquiring digital evidence from the target system.

- Uses automated scripts (Python-based VIGIL-AGENT)
- Extracts:
 - RAM data
 - System logs
 - Disk artifacts
- Supports live forensics (before system shutdown)

This ensures that volatile data is preserved efficiently.

2.2.2 Processing and Hashing Layer

Once data is collected:

- It is processed and converted into SHA-256 hashes
- Each file generates a unique fingerprint
- Even a single-bit change alters the hash completely

This guarantees data integrity verification.

2.2.3 Blockchain (Trust & Immutability Layer)

- Hash values are stored on blockchain using smart contracts (Solidity)
- Provides:
 - Immutability
 - Timestamping
 - Decentralized verification

Blockchain ensures that once recorded, data cannot be altered or deleted.

2.2.4 Storage Layer

- Evidence stored using IPFS (InterPlanetary File System)
- Metadata stored in PostgreSQL
- Ensures:
 - Redundancy
 - Secure distributed storage

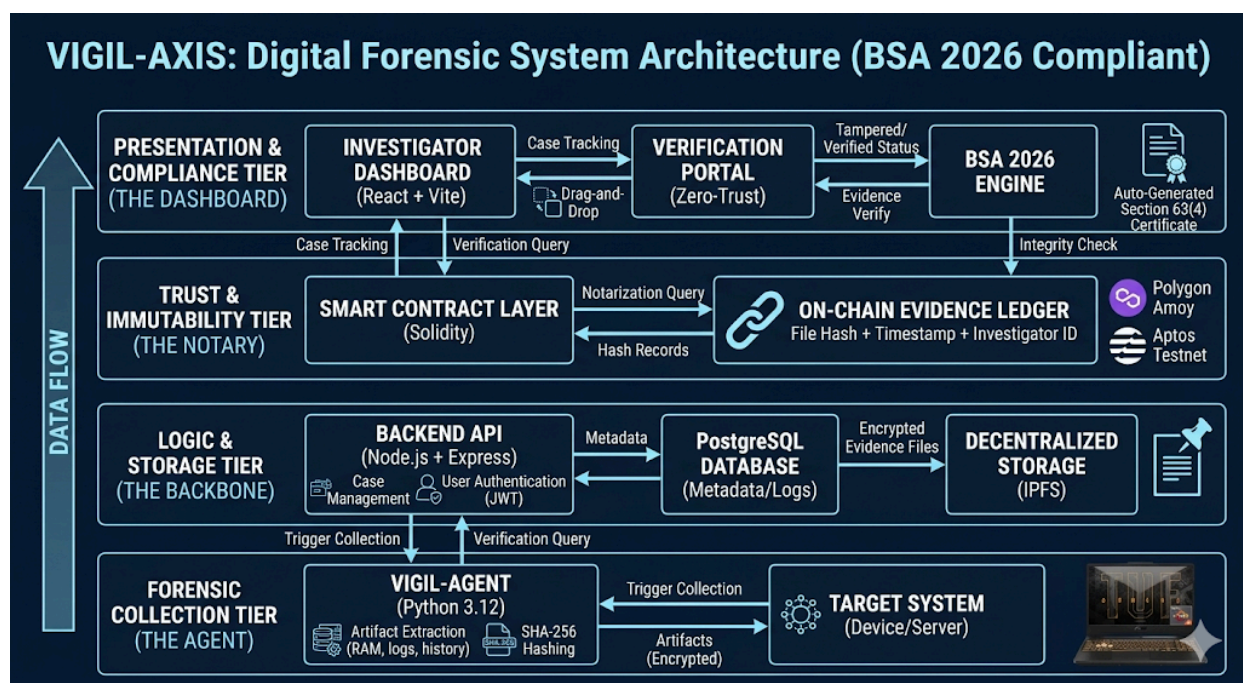
2.2.5 Application Layer

Provides user interaction through:

- Investigator Dashboard
- Verification Engine
- Reporting Module

2.3 Data Flow Diagram (DFD)

The Data Flow Diagram represents how data moves through the system and how different processes interact.



2.3.1 Explanation of DFD Components

The DFD consists of the following key processes:

Process 1: Evidence Collection (1.0)

- Input: Raw artifacts from target system
- Process:
 - Data extraction using VIGIL-AGENT
 - SHA-256 hashing
- Output:
 - Metadata
 - Encrypted evidence

Process 2: Notarization & Anchoring (2.0)

- Hash values are sent to blockchain
- Smart contracts store:
 - Hash
 - Timestamp
 - Investigator ID

This ensures immutability.

Process 3: Secure Storage Management (3.0)

- Evidence stored in IPFS
- Metadata stored in PostgreSQL
- Generates transaction IDs

Process 4: Verification & Reporting (4.0)

- Hash comparison for integrity verification
- Generates:
 - Verification result (Verified/Tampered)
 - Section 63(4) Certificate

External Entities

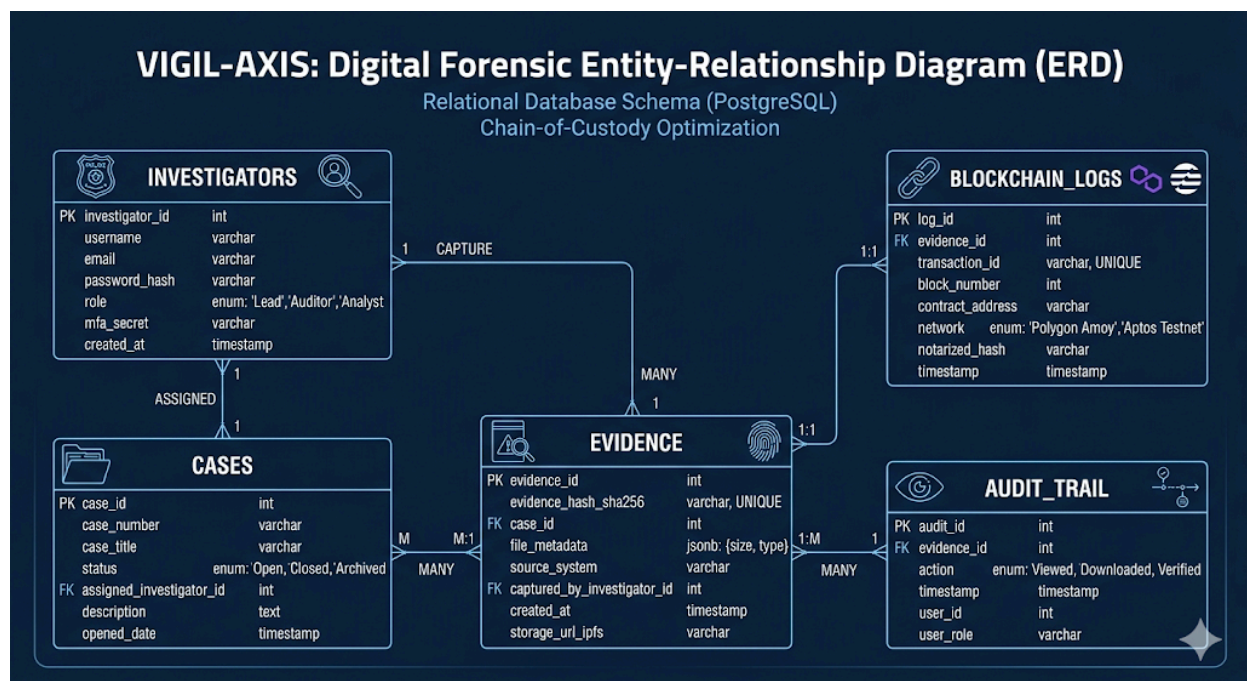
- Target System (source of evidence)
- Investigator Dashboard
- Judicial Portal

Data Stores

- PostgreSQL Database
- IPFS Storage

2.4 Entity Relationship Diagram (ERD)

The ERD defines the structure of the database and relationships between entities.



2.4.1 Entities Description

1. Investigators Attributes:

- investigator_id (Primary Key)
- name
- email
- role

2. Cases Attributes:

- case_id
- case_number
- status
- assigned_investigator

Relationship:

- One investigator → Multiple cases

3. Evidence Attributes:

- evidence_id
- file_hash
- source_system
- timestamp

Relationship:

- One case → Multiple evidence items

4. Blockchain Logs Attributes:

- block_id
- transaction_id
- hash
- timestamp

5. Audit Trail Attributes:

- audit_id
- user_id
- action
- timestamp

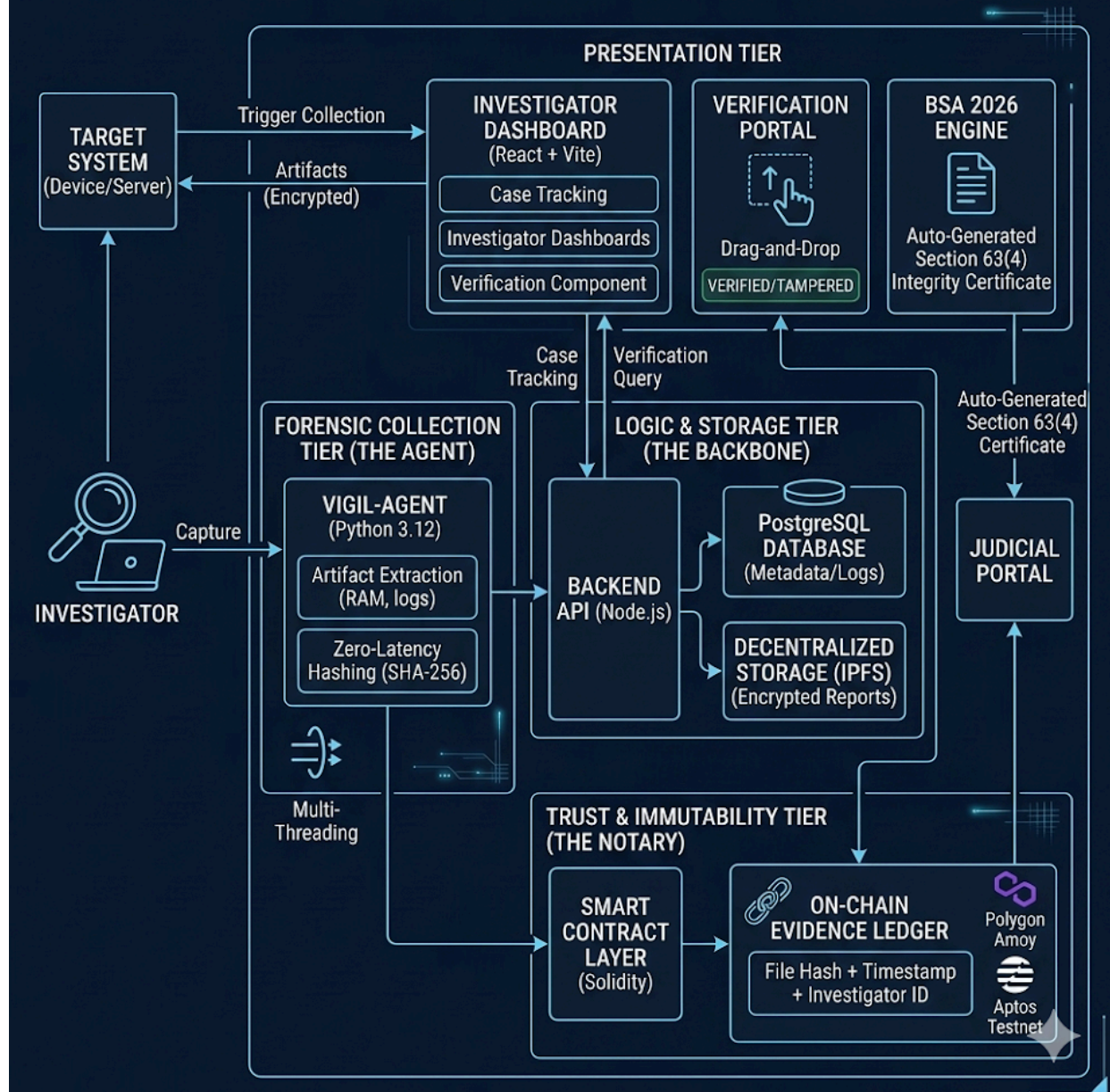
Relationships Summary

- Investigator → Case (1:M)
- Case → Evidence (1:M)
- Evidence → Blockchain Logs (1:1)
- Evidence → Audit Trail (1:M)

2.5 Use Case Diagram

The Use Case Diagram represents system interactions from a user perspective.

VIGIL-AXIS: Use Case Diagram



2.5.1 Actors

1. Investigator
2. System (Automated Agent)
3. Judicial Authority

2.5.2 Use Cases

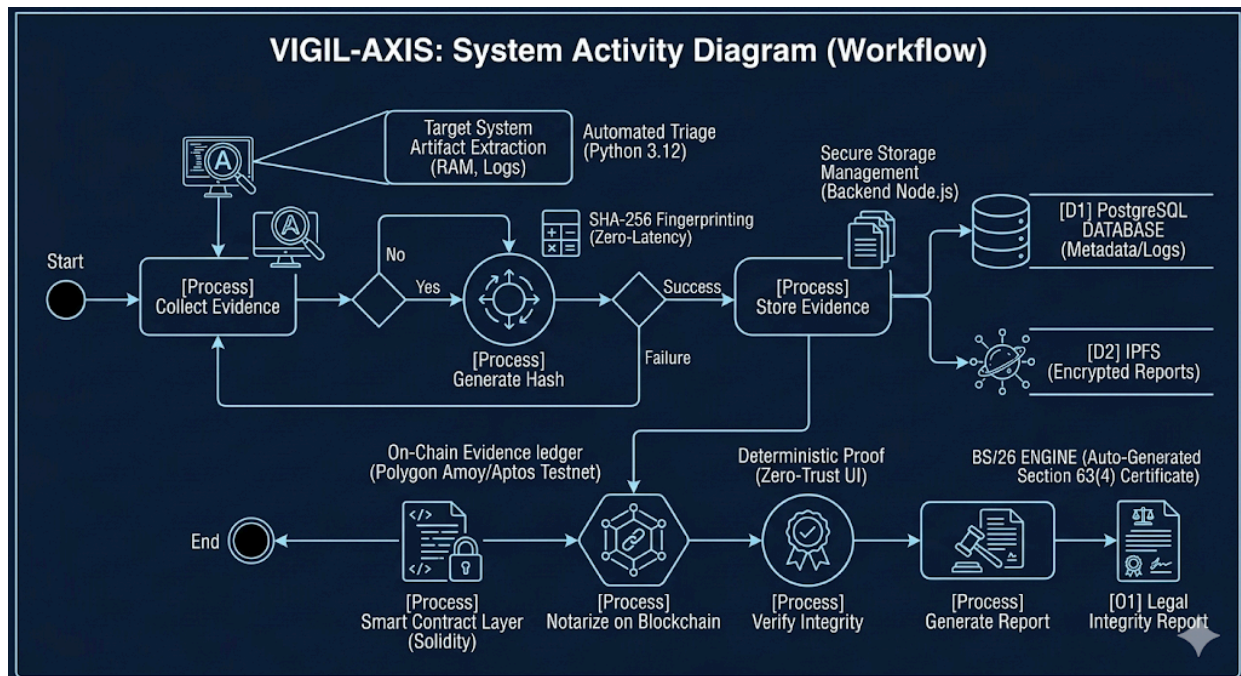
- Upload Evidence
- Collect Evidence (Automated)
- Generate Hash
- Store Evidence
- Verify Evidence
- Generate Report
- View Audit Trail

2.5.3 Description

- Investigator interacts with dashboard
- System performs automated backend operations
- Judicial authority verifies certificate

2.6 Activity Diagram

The Activity Diagram shows the workflow of the system.



2.6.1 Workflow Explanation

1. System initiates evidence collection

2. Data is extracted from target system
3. Hash is generated
4. Hash is stored on blockchain
5. Evidence stored in IPFS
6. User requests verification
7. System compares hash
8. Report and certificate generated

2.7 Design Considerations

The system design is influenced by the following factors:

2.7.1 Security

- Cryptographic hashing
- Blockchain immutability
- JWT authentication

2.7.2 Scalability

- Modular architecture
- Decentralized storage

2.7.3 Reliability

- Redundant storage (IPFS)
- Immutable logs

2.7.4 Legal Compliance

- BSA 2026 alignment
- Automated certificate generation

2.8 Summary

This chapter presented the design of VIGIL-AXIS, including system architecture, data flow, database structure, and workflow representation. The integration of blockchain technology with

forensic processes ensures a secure, transparent, and legally compliant system. The next chapter will focus on the implementation of the system, detailing the technologies and modules used.

CHAPTER 3: IMPLEMENTATION

3.1 Introduction

The implementation phase translates the system design into a functional working model. In the case of VIGIL-AXIS, implementation involves integrating multiple technologies including backend services, frontend interfaces, blockchain infrastructure, and decentralized storage mechanisms. The system is developed as a full-stack application with:

- Backend APIs handling forensic logic
- Frontend dashboards for user interaction
- Blockchain layer ensuring immutability
- IPFS for decentralized storage

The implementation ensures that all modules interact seamlessly to provide a secure and efficient digital forensic platform.

3.2 Development Environment

3.2.1 Software Tools Used

- Backend Framework: Flask / Node.js
- Frontend Framework: React (Vite)
- Database: PostgreSQL
- Blockchain Development: Solidity, Hardhat
- Storage: IPFS
- Version Control: Git

3.2.2 System Configuration

- Processor: Intel i5 or higher
- RAM: 8 GB minimum
- OS: Windows/Linux
- Browser: Chrome/Edge

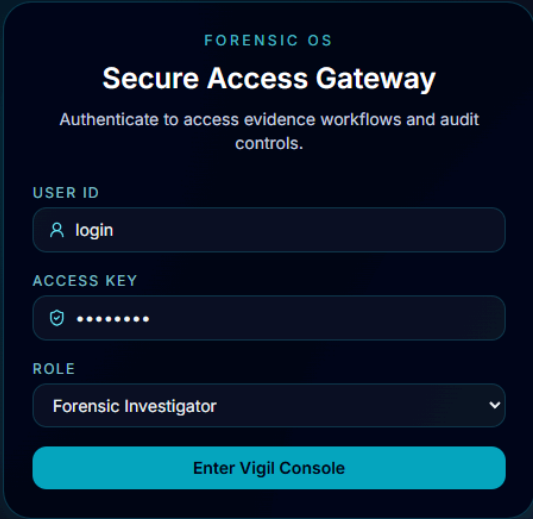
3.3 System Modules

The implementation of VIGIL-AXIS is divided into the following modules:

1. Evidence Collection Module
2. Hash Generation Module
3. Blockchain Integration Module
4. Storage Module
5. Dashboard Module
6. Verification Engine
7. Reporting System
8. Audit Trail Module

3.4 Forensic Investigator

3.4.1 Secure Access Gateway (Login – Investigator Role)



The image shows a login interface for 'FORENSIC OS' titled 'Secure Access Gateway'. The interface is centered on a dark blue background. It includes a subtitle 'Authenticate to access evidence workflows and audit controls.' and three input fields: 'USER ID' with a user icon and the text 'login', 'ACCESS KEY' with a shield icon and seven dots, and 'ROLE' with a dropdown menu showing 'Forensic Investigator'. A large blue button at the bottom is labeled 'Enter Vigil Console'.

FORENSIC OS

Secure Access Gateway

Authenticate to access evidence workflows and audit controls.

USER ID

ACCESS KEY

ROLE

Enter Vigil Console

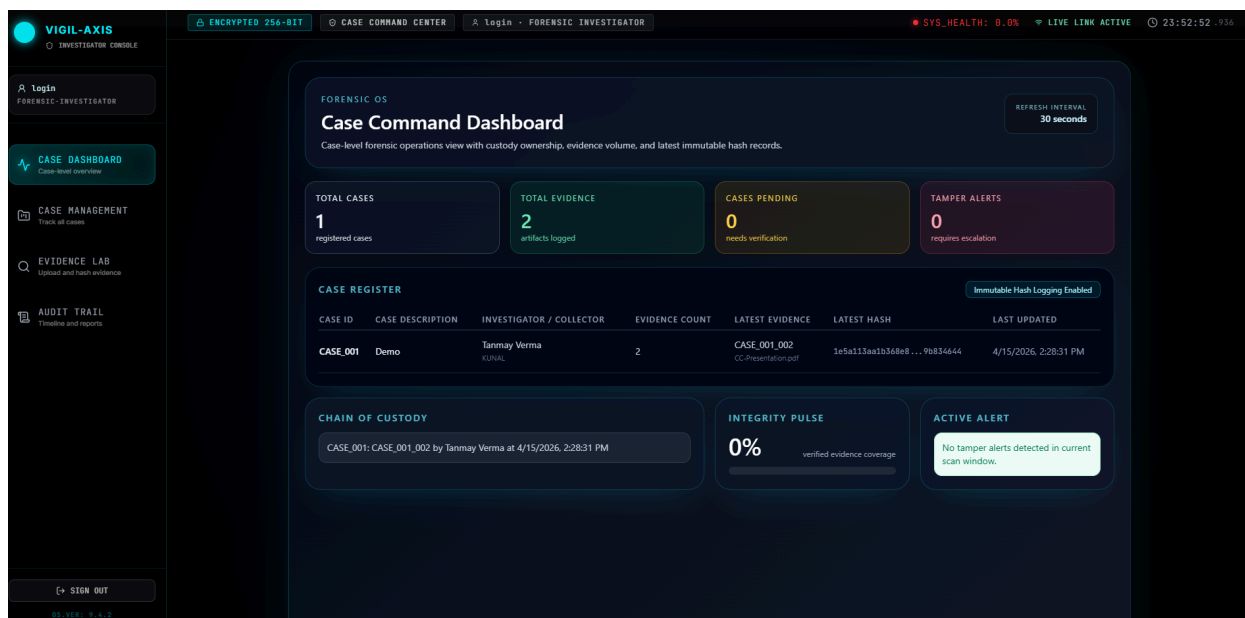
This screen represents the **authentication module** of the VIGIL-AXIS system. It acts as the entry point where users must verify their identity before accessing forensic operations.

The interface requires:

- User ID
- Access Key (password)
- Role selection (Forensic Investigator)

The system uses **secure authentication mechanisms (JWT-based)** to validate users. Role-based access ensures that only authorized personnel can perform sensitive operations such as evidence upload or verification.

3.4.2 Case Command Dashboard (Investigator)

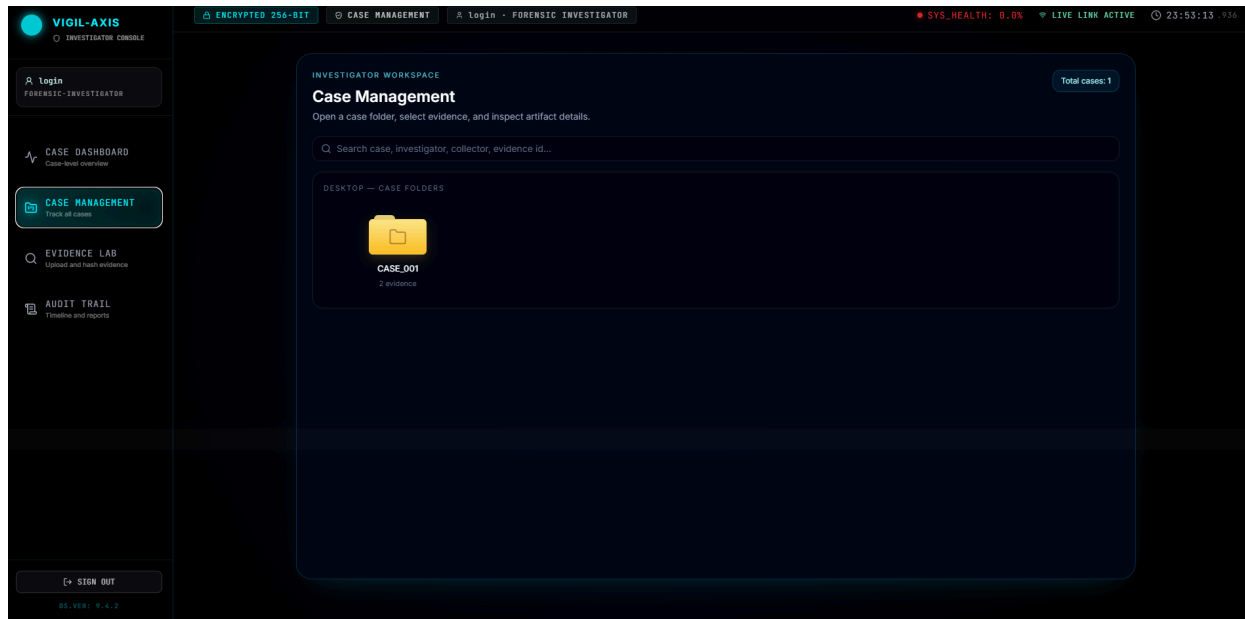


This dashboard provides a **real-time overview of forensic cases**. It displays:

- Total cases
- Evidence count
- Pending verifications
- Tamper alerts

The dashboard acts as a **central control panel**, allowing investigators to monitor ongoing investigations and system health.

3.4.3 Case Management (Folder View)

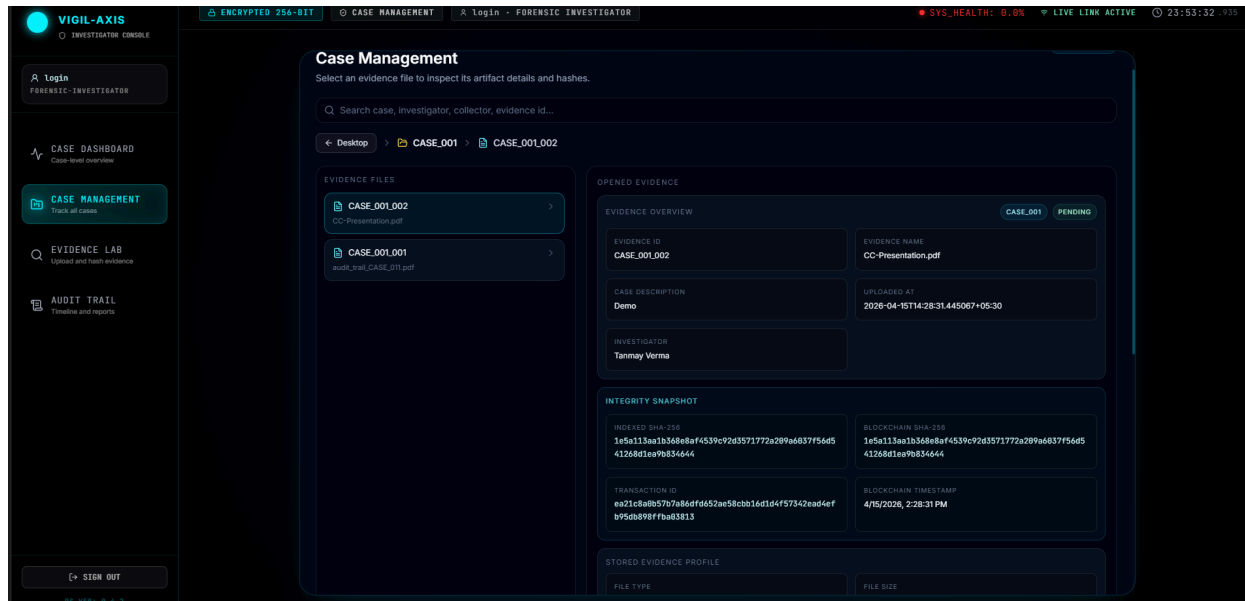


This screen allows users to view **case folders**. Each case is organized like a directory containing multiple evidence files.

It enables:

- Easy navigation
- Organized storage
- Case-wise evidence grouping

3.4.4 Case Management (Detailed View)



This screen displays **detailed evidence information**, including:

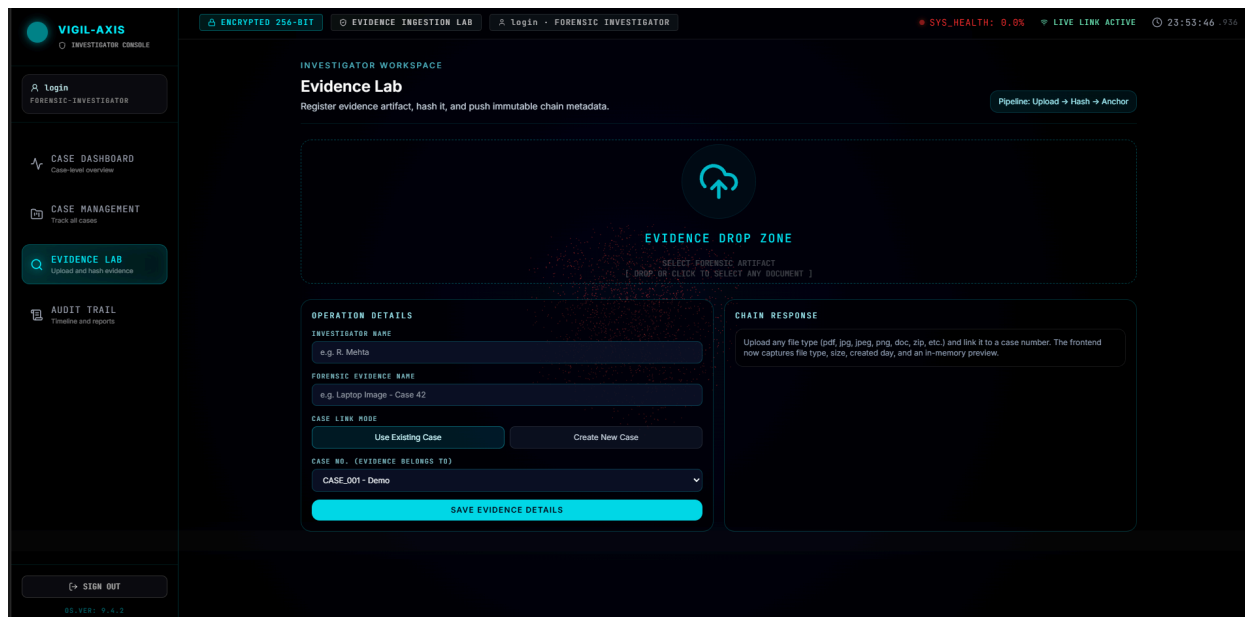
- Evidence ID
- File name
- Investigator
- Timestamp
- SHA-256 hash

It also shows:

- Blockchain hash
- Transaction ID

This ensures **complete traceability of evidence**.

3.4.5 Evidence Lab (Upload Module)

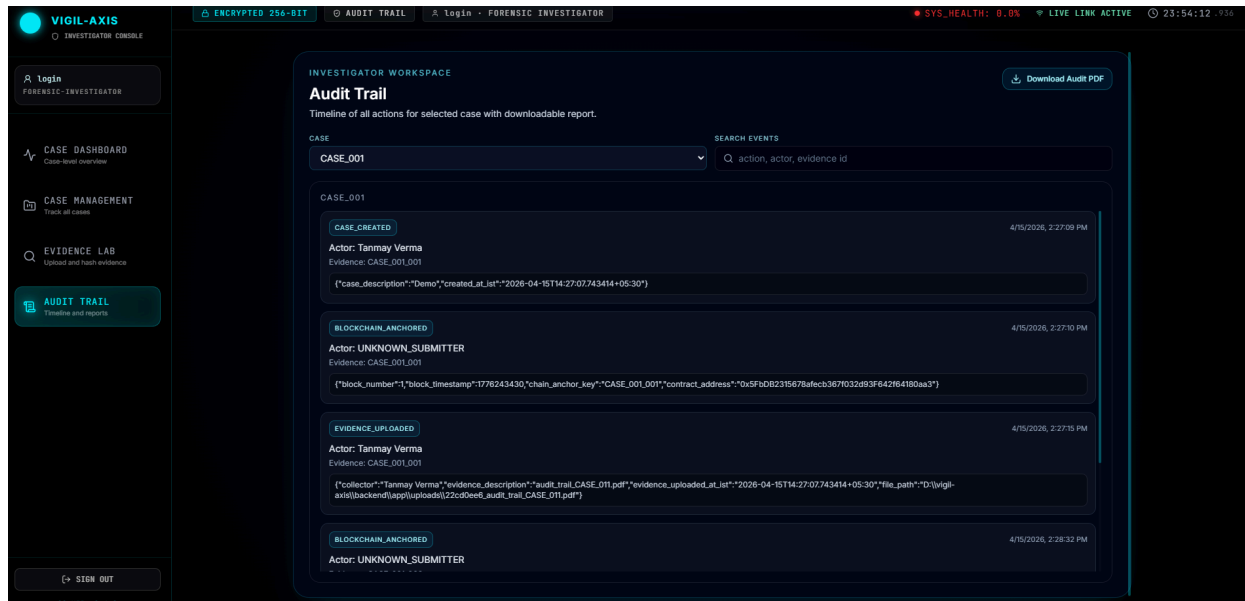


The Evidence Lab is used to:

- Upload forensic artifacts
- Assign evidence to cases
- Trigger hashing and blockchain anchoring

The drag-and-drop interface simplifies evidence ingestion.

3.4.6 Audit Trail



This module maintains a **chronological log of all system activities**, including:

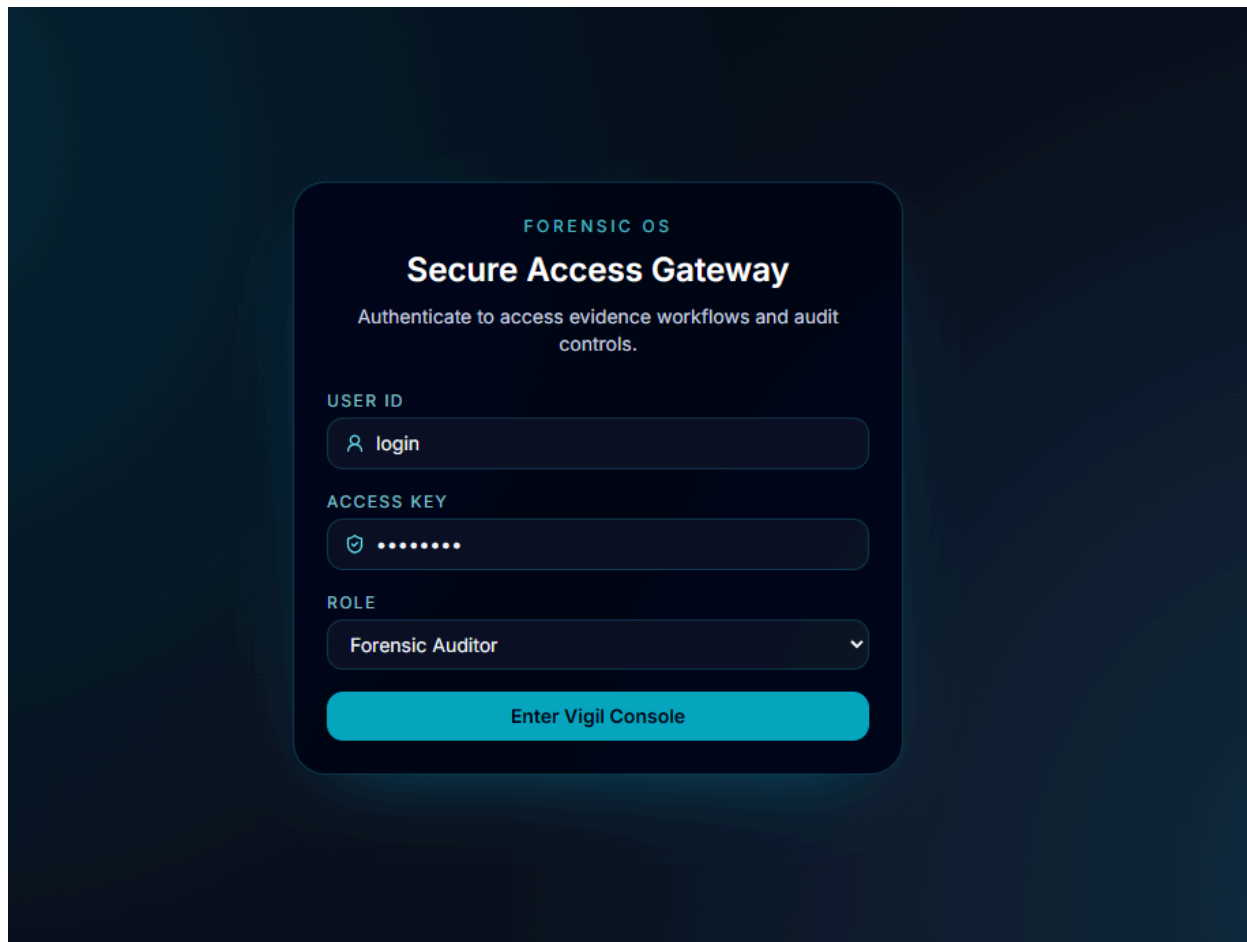
- Case creation
- Evidence upload
- Blockchain anchoring

It ensures:

- Accountability
- Transparency
- Chain of custody tracking

3.5 Forensic Auditor

3.5.1 Secure Access Gateway (Auditor Role)



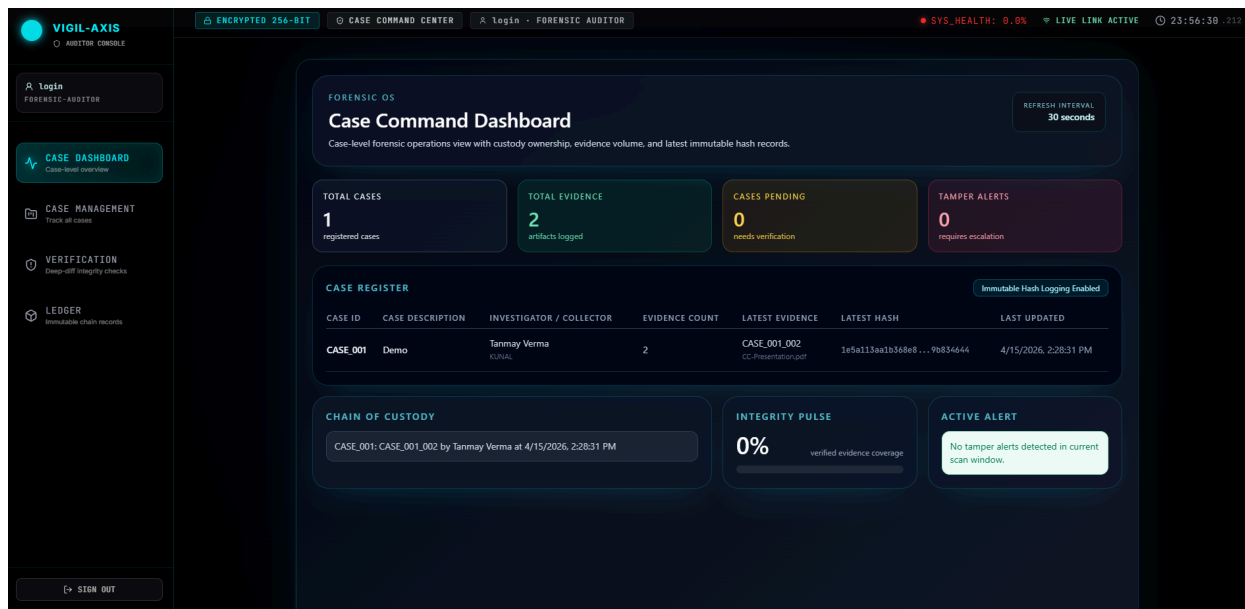
This screen is similar to the investigator login but supports a different role:

- **Forensic Auditor**

Auditors have restricted access and primarily perform:

- Evidence verification
- Report validation

3.5.2 Case Command Dashboard (Auditor)

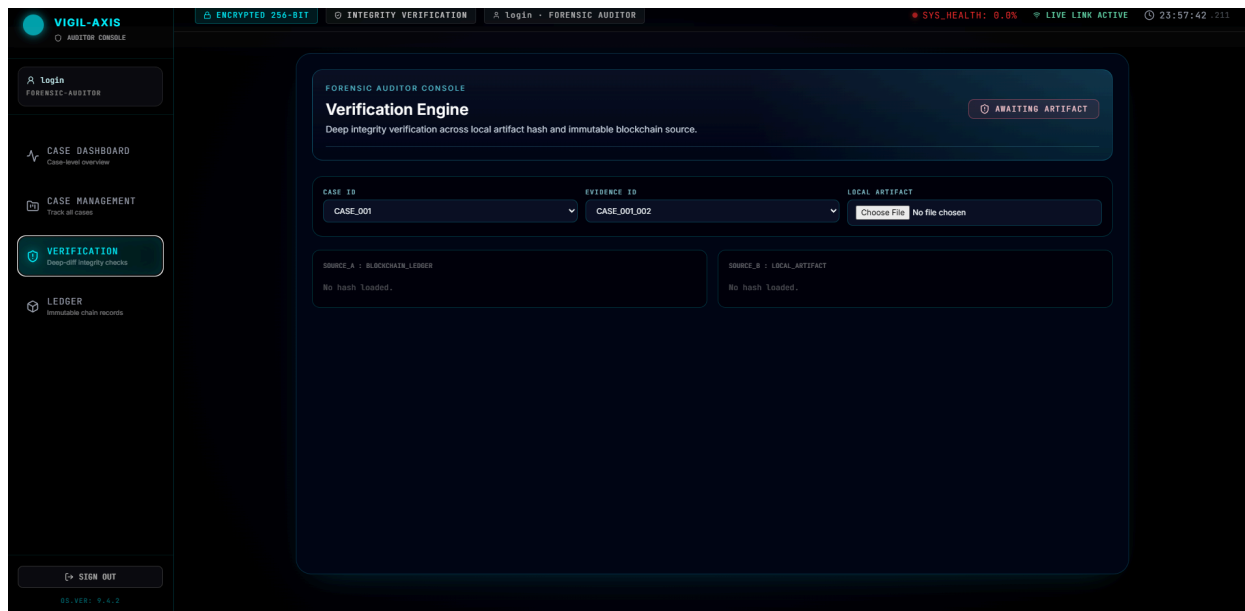


This dashboard provides a **real-time overview of forensic cases**. It displays:

- Total cases
- Evidence count
- Pending verifications
- Tamper alerts

The dashboard acts as a **central control panel**, allowing investigators to monitor ongoing investigations and system health.

3.5.3 Verification Engine (Pending State)



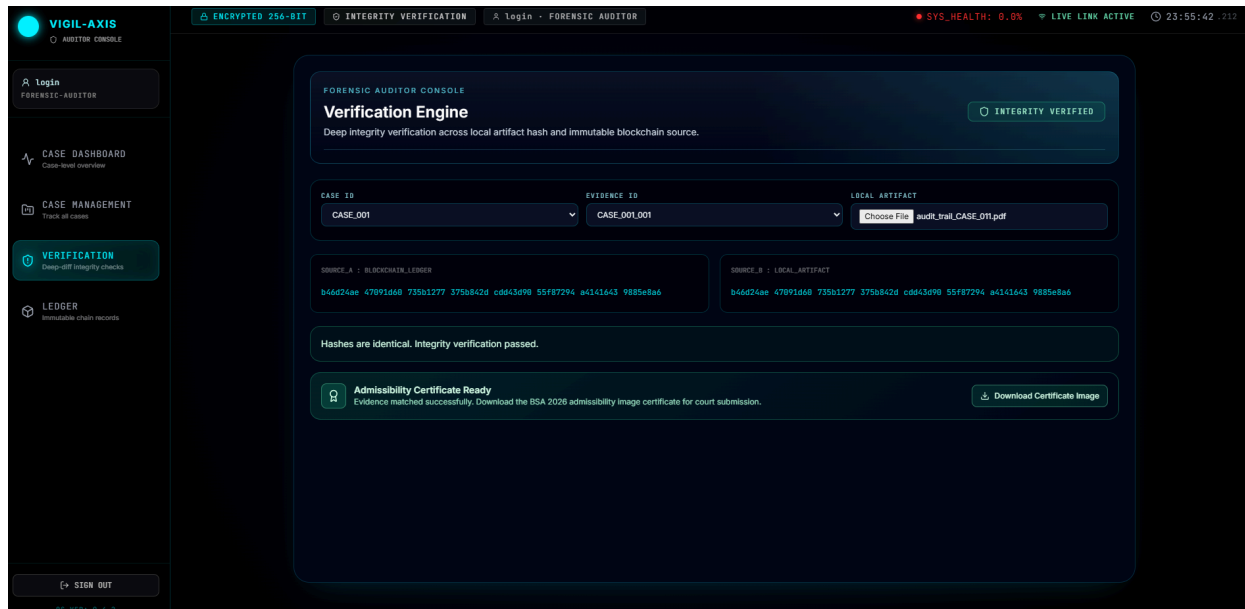
This state occurs when:

- No file is uploaded
- Hash comparison not yet initiated

System shows:

“Awaiting Artifact”

3.5.4 Verification Engine (Successful Case)



This module compares:

- Local file hash
- Blockchain-stored hash

If both match:

Integrity Verified

This confirms that evidence is **untampered and valid**.

3.5.5 BSA 2026 Admissibility Certificate



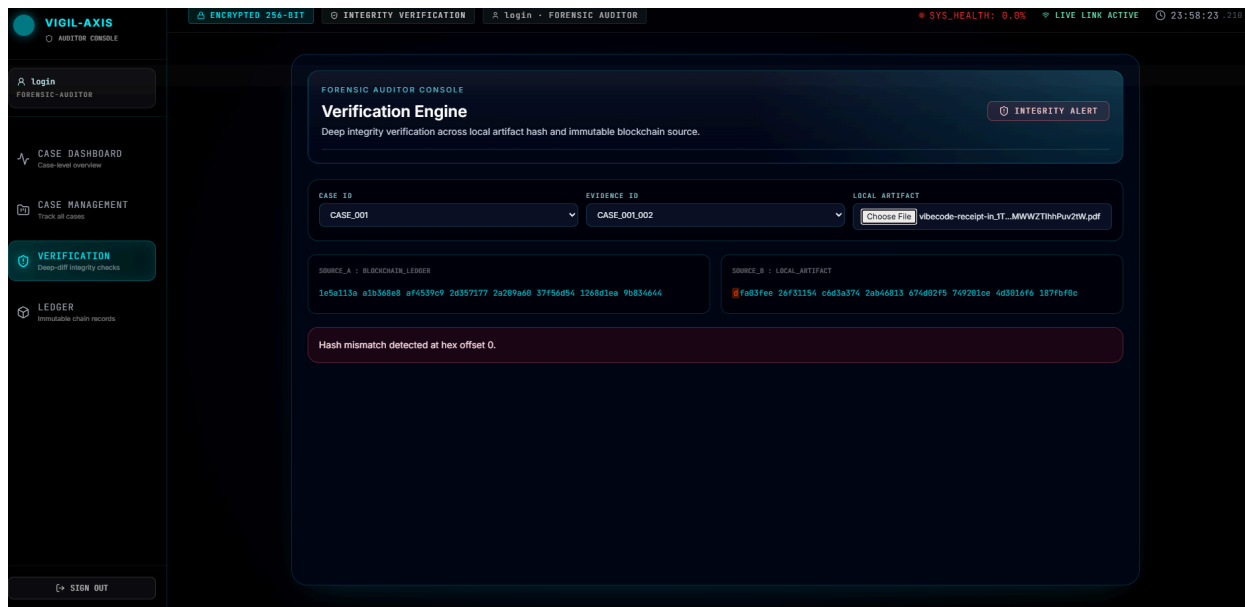
This screen shows the **final legal certificate** generated after verification.

It includes:

- Case ID
- Evidence ID
- Hash values
- Verification result

This certificate is **court-admissible proof of integrity**.

3.5.6 Verification Engine (Tampered Case)



This screen detects tampering.

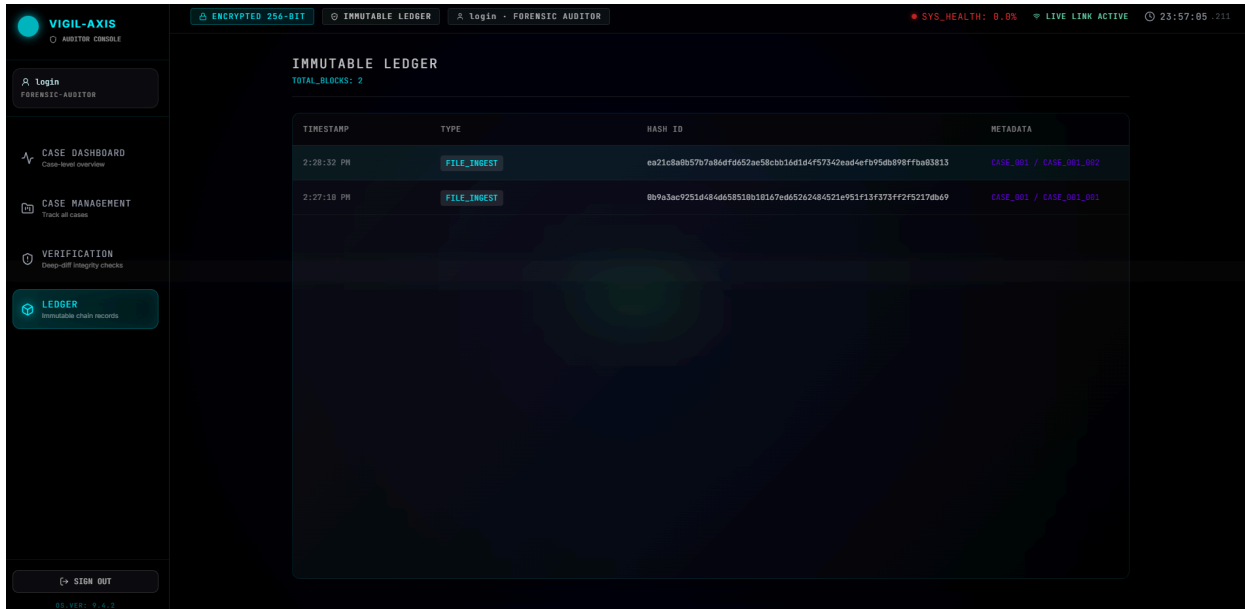
If hashes do not match:

“ **Integrity Alert** “

This indicates:

- Evidence has been modified
- It is not admissible

3.5.7 Immutable Ledger



The screenshot displays the 'IMMUTABLE LEDGER' section of the VIGIL-AXIS application. The interface includes a sidebar with navigation options: Login, CASE DASHBOARD, CASE MANAGEMENT, VERIFICATION, and LEDGER (selected). The main area shows a table of blockchain records with columns for Timestamp, Type, Hash ID, and Metadata. The table contains two entries, both of type 'FILE_INGEST'. The top entry has a timestamp of 2:28:32 PM and a hash ID starting with 'ea21c8a057b7a86df652ae58cbb16d1d4f57342ead4ef95db898f7ba83813'. The bottom entry has a timestamp of 2:27:18 PM and a hash ID starting with '0b9a3ac9251d48d6658518b18167ed65262484521e951f13f373f2f5217db49'. The metadata for both entries is 'CASE_001 / CASE_001_002'. The interface also shows a 'TOTAL_BLOCKS: 2' indicator and a 'SIGN OUT' button in the sidebar.

TIMESTAMP	TYPE	HASH ID	METADATA
2:28:32 PM	FILE_INGEST	ea21c8a057b7a86df652ae58cbb16d1d4f57342ead4ef95db898f7ba83813	CASE_001 / CASE_001_002
2:27:18 PM	FILE_INGEST	0b9a3ac9251d48d6658518b18167ed65262484521e951f13f373f2f5217db49	CASE_001 / CASE_001_001

This screen displays blockchain records:

- Timestamp
- Hash ID
- Case linkage

It ensures:

- Transparency
- Immutable record keeping

3.6 Code Snapshots

3.6.1 Backend Code (Verification Logic)

```
71 def verify_evidence(request):
72     """Orchestrates the comparison of request hashes against the immutable ledger layer.
73     """
74     try:
75         data = request.get_json()
76         if not data:
77             return jsonify({"error": "Missing JSON payload."}), 400
78
79         case_id = data.get('case_id')
80         evidence_id = data.get('evidence_id')
81         user_hash = data.get('hash')
82
83         if not case_id or not user_hash:
84             return jsonify({"error": "Missing fields."}), 400
85
86         case_rows = list_case_evidence_for_case(case_id, limit=500)
87         if not case_rows:
88             return jsonify({"error": "Case ID not found in ledger."}), 404
89
90         resolved_evidence_id = evidence_id or case_rows[0]["evidence_id"]
91         chain_record = get_evidence_record(resolved_evidence_id)
92         evidence_row = get_case_evidence_by_evidence_id(resolved_evidence_id)
93         if not evidence_row:
94             return jsonify({"error": "Evidence not found for case."}), 404
95
96         indexed_record = get_hash_index_record(case_id)
97         stored_hash = chain_record["file_hash"] if chain_record else evidence_row["hash"]
98         status = "VERIFIED" if user_hash == stored_hash else "TAMPERED"
99
100         log_audit_event(
101             action="CASE_VERIFIED" if status == "VERIFIED" else "CASE_TAMPERED",
102             actor="SYSTEM",
103             case_id=case_id,
104             evidence_id=resolved_evidence_id,
105             txid=evidence_row.get("tx_hash") or (indexed_record["txid"] if indexed_record else None),
106             details={
107
```

This code implements:

- Hash comparison logic
- Case validation
- Audit logging

It determines:

if `user_hash == stored_hash` → VERIFIED

else → TAMPERED

3.6.2 Frontend Logic (React Hooks)

```
type ArtifactProfile = {
  fileName: string
  fileType: string
  fileSizeBytes: number
  createdAt: string
  createdDay: string
  cachePath: string
  cacheState: string
  memoryPreviewHex: string
  memoryPreviewSizeBytes: number
}

function generateRandomHex(length: number) {
  let result = ''
  for (let i = 0; i < length; i++) {
    result += HEX_CHARS[Math.floor(Math.random() * HEX_CHARS.length)]
  }
  return result
}

function resolveFileType(file: File) {
  if (file.type) {
    return file.type
  }

  const extension = file.name.split('.').pop()?.toLowerCase() || ''
  return extension || 'unknown'
}

export function useForensics() {
  const [currentHash, setCurrentHash] = useState<string>('0'.repeat(64))
  const [progress, setProgress] = useState<number>(0)
  const [lastUpload, setLastUpload] = useState<UploadEvidenceResponse | null>(null)
  const [lastArtifactProfile, setLastArtifactProfile] = useState<ArtifactProfile | null>(null)
  const [error, setError] = useState<string | null>(null)
```

This code handles:

- File metadata extraction
- State management
- Hash generation

It ensures smooth UI interaction.

3.6.3 Smart Contract (Blockchain Layer)

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract EvidenceRegistry {
5
6     struct Evidence {
7         string caseId;
8         string fileHash;
9         uint256 timestamp;
10        bool exists;
11    }
12
13    // O(1) Mapping for retrieving evidence hashes natively
14    mapping(string => Evidence) private evidenceMap;
15
16    event EvidenceAdded(string indexed caseId, string fileHash, uint256 timestamp);
17
18    function addEvidence(string memory _caseId, string memory _fileHash, uint256 _timestamp) public {
19        // Prevent duplicate caseIds explicitly securely
20        require(!evidenceMap[_caseId].exists, "Evidence for this Case ID already exists");
21
22        evidenceMap[_caseId] = Evidence({
23            caseId: _caseId,
24            fileHash: _fileHash,
25            timestamp: _timestamp,
26            exists: true
27        });
28
29        emit EvidenceAdded(_caseId, _fileHash, _timestamp);
30    }
31
32    function getEvidence(string memory _caseId) public view returns (string memory caseId, string memory fileHash, uint256 timestamp) {
33        require(evidenceMap[_caseId].exists, "Evidence not found for the given Case ID");
34        Evidence memory ev = evidenceMap[_caseId];
35        return (ev.caseId, ev.fileHash, ev.timestamp);
36    }
37 }
```

This Solidity code:

- Stores evidence hash
- Prevents duplication
- Emits blockchain events

Key features:

- Immutable storage
- Transparent verification
- Secure mapping of case → hash

3.7 Security Implementation

Security features are integrated at all layers of the system.

Implementation Details

- **Authentication:** JWT (JSON Web Tokens) are used for securing API communication.

- **Integrity:** SHA-256 hashing is mandatory for all evidence.
- **Immutability:** Smart contracts prevent alteration of forensic records.
- **Data Protection:** Evidence is encrypted before being stored on IPFS.

3.8 System Workflow Implementation

The system workflow is implemented as follows:

9. Evidence is collected from the target system
10. Hash is generated
11. Hash is stored on blockchain
12. Evidence stored in IPFS
13. Metadata stored in database
14. User verifies evidence
15. System generates report

3.9 Integration of Modules

All modules are integrated through REST APIs, ensuring seamless communication between frontend, backend, and blockchain layers.

3.10 Security Implementation

Security is implemented at multiple levels:

- SHA-256 hashing for integrity
- Blockchain for immutability
- JWT for authentication
- Encryption for storage

3.11 Challenges Faced

- Blockchain integration complexity
- Handling large forensic data
- Ensuring real-time performance

3.12 Summary

The implementation of VIGIL-AXIS successfully integrates multiple technologies into a unified forensic system. The use of blockchain ensures data integrity, while automated workflows enhance efficiency and reliability.

CHAPTER 4: TESTING

4.1 Introduction

Testing is a critical phase in the software development lifecycle that ensures the correctness, reliability, and performance of a system. For a digital forensic system such as VIGIL-AXIS, testing becomes even more crucial due to its direct involvement in legal proceedings. Any failure in the system can lead to incorrect evidence validation, which may have serious judicial consequences. The primary objective of testing in VIGIL-AXIS is to verify that:

- Evidence is collected accurately without loss
- Hash values are generated correctly
- Blockchain storage is immutable
- Verification results are reliable
- Reports comply with legal standards (BSA 2026)

The testing process is divided into multiple levels, including unit testing, integration testing, system testing, and security testing, ensuring comprehensive validation of the system.

4.2 Testing Objectives

The key objectives of testing the VIGIL-AXIS system are:

1. To verify the functionality of each module independently
2. To ensure proper integration between modules
3. To validate system performance under different conditions
4. To ensure data integrity and immutability
5. To confirm compliance with legal requirements
6. To detect and eliminate bugs and vulnerabilities

4.3 Types of Testing Performed

The following types of testing were conducted:

- Unit Testing
- Integration Testing
- System Testing
- Functional Testing

- Performance Testing
- Security Testing
- User Acceptance Testing (UAT)

4.4 Unit Testing

4.4.1 Overview

Unit testing focuses on testing individual components of the system in isolation. Each module is tested independently to ensure it performs its intended function correctly.

4.4.2 Unit Testing of Modules

1. Evidence Collection Module

Test Case ID	Input	Expected Output	Result
TC01	Upload file	File successfully uploaded	Pass
TC02	Upload invalid file	Error message displayed	Pass

2. Hash Generation Module

Test Case ID	Input	Expected Output	Result
TC03	Valid file	SHA-256 hash generated	Pass
TC04	Modified file	Different hash generated	Pass

3. Blockchain Module

Test Case ID	Input	Expected Output	Result
TC05	Store hash	Transaction recorded	Pass
TC06	Retrieve hash	Correct hash displayed	Pass

4. Verification Module

Test Case ID	Input	Expected Output	Result
TC07	Matching hash	Verified	Pass

TC08	Non-matching hash	Tampered	Pass
------	-------------------	----------	------

4.5 Integration Testing

4.5.1 Overview

Integration testing verifies the interaction between different modules to ensure seamless data flow.

4.5.2 Integration Scenarios

Scenario 1: Collection -> Hashing -> Blockchain

Step	Expected Result
Upload evidence	File accepted
Generate hash	Hash created
Store on blockchain	Transaction successful

Scenario 2: Storage -> Verification -> Reporting

Step	Expected Result
Retrieve file	File loaded
Verify integrity	Correct result
Generate report	Certificate generated

4.5.3 Results

All modules were successfully integrated, and data flow between components was consistent and error-free.

4.6 System Testing

4.6.1 Overview

System testing evaluates the complete system as a whole to ensure it meets functional and non-functional requirements.

4.6.2 Functional Testing

Test Case ID	Function	Expected Result	Result
TC09	Upload evidence	Successful upload	Pass
TC10	Verify evidence	Correct result	Pass
TC11	Generate report	Report created	Pass

4.6.3 Non-Functional Testing

Performance Testing

- Response time: < 2 seconds
- Blockchain transaction delay: acceptable

Usability Testing

- Interface is user-friendly
- Navigation is intuitive

Reliability Testing

- System stable under repeated operations

4.7 Security Testing

4.7.1 Overview

Security testing ensures that the system is protected against unauthorized access and data manipulation.

4.7.2 Security Measures Tested

- Data encryption
- Authentication (JWT)

- Blockchain immutability
- Access control

4.7.3 Test Results

Test Case ID	Scenario	Expected Result	Result
TC12	Unauthorized access	Access denied	Pass
TC13	Data tampering attempt	Detected	Pass

4.8 Performance Testing

4.8.1 Overview

Performance testing evaluates system efficiency under different workloads.

4.8.2 Metrics

- Response Time
- Throughput
- System Load Handling

4.8.3 Results

- System handled multiple requests efficiently
- No significant delays observed
- Stable performance under load

4.9 User Acceptance Testing (UAT)

4.9.1 Overview

UAT ensures that the system meets user expectations and requirements.

4.9.2 Feedback

- Easy to use dashboard
- Accurate verification results
- Useful reporting features

4.10 Test Data and Results

Sample Test Data

- File: memory_dump.raw
- File: system_log.txt

Observed Results

File	Hash Generated	Verification
memory_dump.raw	Valid hash	Verified
system_log.txt	Valid hash	Verified

4.11 Error Handling and Debugging

The system includes mechanisms for:

- Handling invalid inputs
- Displaying error messages
- Logging system errors

4.12 Limitations of Testing

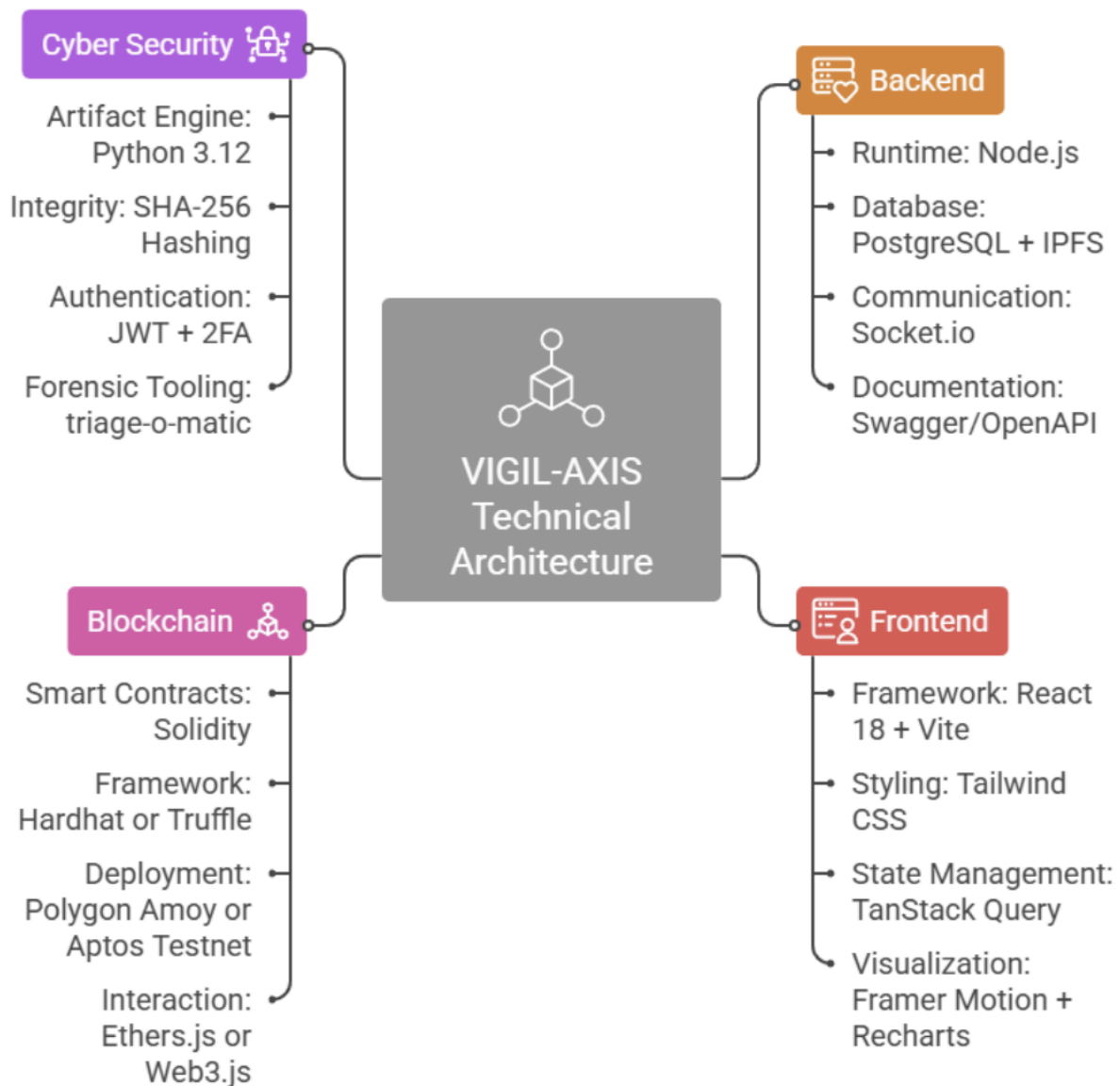
- Limited real-world data testing
- Blockchain network dependency
- Performance varies with network latency

4.13 Summary

The testing phase confirms that VIGIL-AXIS is a reliable and secure digital forensic system. All modules performed as expected, and the system successfully met its functional and non-functional requirements. The integration of blockchain ensures tamper-proof evidence handling, making the system suitable for legal applications.

CHAPTER 5: Technical Stack & Innovation

The tech stack includes Python 3.12 for the artifact engine, React 18 for the Mission Control UI, and Polygon/Aptos for the blockchain ledger. Core innovations include **Deterministic Integrity** (mathematical proof), a native **BSA 2026 Engine** for automated compliance, and a **Zero-Trust System** providing dashboard validation against the distributed ledger.



Financial Analysis

The project targets a Total Addressable Market (TAM) of \$1.273 Trillion globally, with a Serviceable Obtainable Market (SOM) estimated at \$9.6 Billion by 2030, leveraging a B2G/B2B compliance-first model.

CHAPTER 6: FINANCIAL ANALYSIS

6.1 Introduction

Financial analysis plays a crucial role in evaluating the economic feasibility, sustainability, and scalability of a software system. For a project like VIGIL-AXIS, which integrates blockchain technology with digital forensics, it is essential to assess both the cost structure and revenue potential. The system is designed as a Forensic-as-a-Service (FaaS) platform targeting government agencies, law enforcement organizations, and corporate cybersecurity teams. The financial model focuses on balancing operational costs with scalable revenue streams while ensuring affordability and compliance with legal standards such as BSA 2026.

6.2 Cost Structure

The cost structure of VIGIL-AXIS is divided into initial setup costs and operational costs.

6.2.1 Initial Setup

Cost Component	Cost (INR)
Development Tools & Software	₹2,000
Blockchain Deployment Setup	₹3,000
Testing & Debugging	₹2,000
Miscellaneous	₹3,000
Total Initial Cost	₹10,000

6.2.2 Operational Cost per Case

Component	Cost (INR)
Agent Setup Fee	₹5,000
Evidence Processing (₹100 × 100 artifacts)	₹2,000

Secure Storage (1 Year)	₹3,000
IPFS Pinning & Hash Registry	₹2,000
SHA-256 Hashing & Verification	₹1,000
Total Operational Cost	₹13,000

6.3 Revenue Model

The revenue model of VIGIL-AXIS is based on a per-case transaction fee.

- Client Fee per Case: ₹15,000
- Operational Cost: ₹13,000
- Profit per Case: ₹2,000

This pricing ensures:

- Competitive market positioning
- Cost recovery with profit margin
- Scalability for large deployments

6.4 Market Analysis

The potential market for VIGIL-AXIS is evaluated using three key metrics:

6.4.1 Total Addressable Market (TAM)

Represents the global digital forensic market size.

- Estimated Value: ₹1,273 Billion (FY 2024)

6.4.2 Serviceable Available Market (SAM)

Represents the Indian digital forensic market.

- Estimated Value: ₹118 Billion (FY 2026)

6.4.3 Serviceable Obtainable Market (SOM)

Represents the realistic share VIGIL-AXIS can capture.

- Estimated Value: ₹9.6 Billion by 2030

6.5 Profitability Analysis

The profitability of the system depends on the number of cases processed.

Example Calculation

Number of Cases	Revenue (₹15,000 each)	Cost (₹13,000 each)	Profit
100 Cases	₹15,00,000	₹13,00,000	₹2,00,000
500 Cases	₹75,00,000	₹65,00,000	₹10,00,000
1000 Cases	₹15,00,00,000	₹13,00,00,000	₹20,00,00,000

This demonstrates that the system becomes highly profitable with increased adoption.

6.6 Growth Projection

The projected growth of VIGIL-AXIS is based on increasing demand for digital forensic solutions.

- Expected Growth Rate: 40% CAGR
- Timeline: 12–36 months deployment cycle

Growth Phases

1. Initial deployment (pilot projects)
2. Expansion to law enforcement agencies
3. Adoption by corporate sector
4. Nationwide scaling

6.7 Cost-Benefit Analysis

Benefits

- Reduced manual effort
- Faster evidence processing
- Improved legal compliance
- Increased trust in digital evidence

Costs

- Blockchain transaction fees

- Storage costs
- Infrastructure maintenance

6.8 Break-Even Analysis

The break-even point is achieved when total revenue equals total costs.

- Break-even occurs after approximately 50–100 cases, depending on operational efficiency.

6.9 Financial Feasibility

The project is financially feasible due to:

- Low initial investment
- Recurring revenue model
- High scalability
- Increasing demand for cybersecurity solutions

6.10 Summary

The financial analysis of VIGIL-AXIS demonstrates that the system is economically viable and scalable. With a strong revenue model, growing market demand, and efficient cost management, the project has the potential to achieve sustainable profitability. The integration of blockchain technology further enhances its value proposition, making it a competitive solution in the digital forensic domain.

CHAPTER 7: CONCLUSION

7.1 Introduction

The rapid evolution of cyber threats has necessitated the development of robust, secure, and legally compliant digital forensic systems. Traditional forensic tools, while effective in data acquisition and analysis, often fall short in ensuring data integrity, transparency, and legal admissibility. The need for a system that bridges the gap between technical investigation and judicial requirements has become increasingly critical.

The project VIGIL-AXIS: The Immutable Forensic OS was conceived to address these challenges by integrating automation, cryptographic security, and blockchain technology into a unified platform.

7.2 Summary of the Work

The primary objective of this project was to design and implement a system capable of:

- Automating forensic evidence collection
- Ensuring data integrity using SHA-256 hashing
- Providing immutable storage through blockchain
- Maintaining a secure chain of custody
- Generating legally compliant certificates under BSA 2026

The system was successfully developed as a full-stack application, incorporating:

- A responsive frontend dashboard
- Backend APIs for processing and logic
- Blockchain integration for notarization
- IPFS for decentralized storage

The integration of these components resulted in a system capable of transforming digital forensic processes into a verifiable and transparent framework.

7.3 Achievement of Objectives

The objectives outlined at the beginning of the project were effectively achieved:

Automation of Evidence Collection

The system successfully implements automated scripts that collect forensic data from live systems, minimizing data loss and human intervention.

Data Integrity Assurance

The use of SHA-256 hashing ensures that each piece of evidence is uniquely identified and any modification is easily detectable.

Blockchain-Based Immutability

By storing hash values on a blockchain, the system guarantees that once recorded, data cannot be altered, ensuring trust and transparency.

Chain of Custody Management

A comprehensive audit trail tracks all interactions with evidence, maintaining a continuous and verifiable chain of custody.

Legal Compliance

The system supports the generation of Section 63(4) certificates, aligning with the requirements of BSA 2026.

7.4 Key Contributions

The project contributes significantly to the field of digital forensics in the following ways:

1. **Integration of Blockchain with Digital Forensics** Provides a novel approach to ensuring immutability and trust.
2. **Automation of Forensic Processes** Reduces manual effort and increases efficiency.
3. **Legal Compliance Framework** Bridges the gap between technical investigation and legal requirements.
4. **Unified Platform** Combines evidence collection, storage, verification, and reporting.
5. **Enhanced Transparency** Enables stakeholders to verify evidence independently.

7.5 Advantages of the System

- Tamper-proof evidence handling

- Real-time data acquisition
- Reduced human error
- High scalability
- Improved trust in judicial processes

7.6 Limitations

Despite its strengths, the system has certain limitations:

- Dependency on blockchain network performance
- Limited testing with real-world large-scale datasets
- Requirement of technical knowledge for deployment
- Storage costs associated with decentralized systems

7.7 Conclusion

In conclusion, VIGIL-AXIS successfully addresses critical challenges in digital forensics by providing a secure, automated, and legally compliant solution. The integration of blockchain technology ensures that digital evidence is not only preserved but also verifiable beyond doubt.

The system represents a significant step towards modernizing forensic investigations and strengthening the credibility of digital evidence in courts of law.

CHAPTER 8: FUTURE SCOPE

8.1 Introduction

While VIGIL-AXIS provides a robust foundation for secure digital forensic operations, there are numerous opportunities for further enhancement and expansion. The rapid advancement of technology presents new possibilities for improving system capabilities, scalability, and usability.

8.2 Technical Enhancements

8.2.1 AI-Based Evidence Analysis

Future versions of the system can integrate Artificial Intelligence and Machine Learning techniques to:

- Detect anomalies in data
- Identify suspicious patterns
- Automate forensic analysis

8.2.2 Advanced Blockchain Integration

- Implementation of Layer-2 scaling solutions
- Reduction in transaction costs
- Faster processing times

8.2.3 Multi-Chain Support

Supporting multiple blockchain networks to enhance:

- Flexibility
- Interoperability
- Redundancy

8.2.4 Enhanced Security Features

- Multi-factor authentication
- Zero-trust architecture
- Advanced encryption techniques

8.3 Functional Enhancements

8.3.1 Mobile Application Development

- Mobile-based evidence collection
- Real-time monitoring
- Remote verification

8.3.2 Cloud Integration

- Scalable infrastructure
- Improved accessibility
- Cost optimization

8.3.3 Real-Time Collaboration

- Multi-user access
- Team-based investigation workflows
- Shared dashboards

8.4 Legal and Compliance Expansion

- Adaptation to international legal standards
- Integration with global forensic frameworks
- Support for cross-border investigations

8.5 Industry Applications

The system can be extended to:

- Corporate cybersecurity
- Financial fraud detection
- Government intelligence agencies
- Healthcare data security

8.6 Research Opportunities

Future research can focus on:

- Blockchain optimization

- Forensic data compression
- Privacy-preserving forensic techniques

8.7 Conclusion

The future scope of VIGIL-AXIS is vast, with potential applications across multiple domains. By incorporating advanced technologies and expanding its capabilities, the system can evolve into a comprehensive digital forensic ecosystem.

CHAPTER 9: REFERENCES

9.1 Introduction

This chapter lists the sources and references used during the development of the project. These references include research papers, books, online resources, and technical documentation.

9.2 Books

6. 1. Stallings, William – Cryptography and Network Security
7. 2. Nelson, Phillips – Guide to Computer Forensics and Investigations
8. 3. Kurose, James – Computer Networking: A Top-Down Approach

9.3 Research Papers

9. 1. “Blockchain-Based Digital Forensics Framework” – IEEE
10. 2. “Secure Evidence Management using Blockchain Technology”
11. 3. “Digital Chain of Custody in Cyber Forensics”

9.4 Websites

- <https://ethereum.org>
- <https://ipfs.io>
- <https://flask.palletsprojects.com>
- <https://react.dev>

9.5 Tools and Documentation

- Hardhat Documentation
- PostgreSQL Documentation
- Node.js Documentation
- JWT Authentication Guide

9.6 Standards and Legal References

- Bharatiya Sakshya Adhiniyam (BSA) 2026

- Digital Forensics Standards (NIST)
- ISO/IEC 27037 Guidelines

9.7 Conclusion

The references listed above provided the foundational knowledge and technical guidance required for the successful development of the VIGIL-AXIS system.